Sistema Integrado de Monitoreo, Alertas Georreferenciadas y Despacho CAT para la Gestión de Seguridad Municipal, tipo B

(Sistema Integrado CAD)

PLAN DE ARQUITECTURA DE SOFTWARE

MVP académico con datos simulados

Fecha: 04/10/2026

Versión: 1.0 (propuesta para revisión del equipo)

Fuentes: Plan Scrum Sistema Integrado CAD (actualizado) · Historias de usuario estructuradas · Respuestas del equipo del 04/10/2026

Control de versiones

Versión	Fecha	Descripción
1.0	04/10/2026	Primera versión: diagnóstico (fase 1) incorporado, arquitectura, stack, diagramas, ADR y plan de implementación. Pendiente de revisión por el equipo.

Índice

Si el índice aparece vacío, en Word use clic derecho → "Actualizar campo" (o F9).

Resumen ejecutivo

Este documento define una arquitectura **implementable por un equipo de tres personas** para el MVP académico del *Sistema Integrado de Monitoreo, Alertas Georreferenciadas y Despacho CAT* (en adelante, **Sistema Integrado CAD**). Parte de los requisitos y restricciones documentados en el Plan Scrum y en las historias de usuario, y de las respuestas del equipo del 04/10/2026. Hoy no existe código ni repositorio: todo lo descrito es una propuesta.

Arquitectura elegida. Un **monolito modular**: un único proceso **ASP.NET Core 10** (el *Servidor CAD*) expone la API REST, el canal de tiempo real (**SignalR** sobre WebSocket) y las páginas del **panel web de central**, y ejecuta un temporizador interno para las reglas que dependen del tiempo. **PostgreSQL** es la única fuente de verdad. Ciudadano y sereno usan una **aplicación móvil nativa para Android construida con .NET MAUI 10**, que comparte lenguaje y contratos con el servidor. Los mapas usan **OpenStreetMap** (Leaflet en el panel y Mapsui en la app).

Motivos principales.

- La validación exige **20 sesiones concurrentes** (RNF03): una sola instancia basta, y servicios separados, brokers o cachés distribuidas no aportan nada verificable.
- Las garantías críticas (**una asignación vigente por caso y por sereno, una sesión de acompañamiento activa por ciudadano, sin duplicados ante reintentos**, RNF08) se resuelven con transacciones, restricciones únicas parciales y control optimista de concurrencia de PostgreSQL, sin infraestructura adicional.
- El objetivo de **≤3 s para el 95 % de los SOS** (RNF02) requiere notificar sin sondeo: SignalR, incluido en ASP.NET Core, lo cubre en el mismo proceso.
- El equipo trabajará en **C#/.NET**: servidor, app y pruebas comparten un solo lenguaje y se reduce la curva de aprendizaje.
- Con el **presupuesto mínimo**: Render (gratuito en desarrollo y plan Starter, unos USD 7/mes prorrateados por segundo, solo durante la validación y las demostraciones) y Neon PostgreSQL gratuito. Así se evita la suspensión por inactividad durante las pruebas de RNF02 y RNF10, y la caducidad a los 30 días de la base gratuita de Render.

Cambio relevante respecto del Plan Scrum. El equipo indicó el 04/10/2026 que el entorno móvil debe ser una **aplicación**, preferentemente con framework nativo, en lugar de la PWA prevista en [PS] 1.4. El documento lo registra como decisión (ADR-01) y señala qué partes del Plan Scrum y de RNF04 conviene actualizar.

Pendientes que condicionan la aceptación. No hay ninguna de las decisiones DP01–DP22 resuelta. La arquitectura las trata como parámetros configurables o como funcionalidades bloqueadas, sin inventar reglas. El caso más crítico es **DP22** (plazo de conservación de recorridos): sin él no puede aprobarse RNF13 ni HU39-B. Además, el calendario de [PS] ya no se corresponde con el estado real (sin código a 25 días de la entrega prevista del 29/10/2026). La replanificación del alcance corresponde al Product Owner.

0. Fuentes, convenciones y alcance del documento

0.1. Fuentes utilizadas

Tabla 1. Fuentes del análisis

Código	Fuente	Uso en este documento
[PS]	Plan Scrum Sistema Integrado CAD – actualizado (.docx)	Objetivo, alcance, RF01.1–RF23.2, RNF01–RNF14, restricciones, backlog, sprints
[HU]	Historias de usuario estructuradas (.docx)	HU01–HU43, criterios de aceptación (CA), referencias a DP
[EQ]	Respuestas del equipo en la conversación del 04/10/2026	Estado (sin código), C#/.NET, presupuesto mínimo, entorno controlado aceptable, Android/Windows con Chrome, app móvil nativa, DP sin resolver
[DOC]	Documentación oficial de proveedores, consultada el 04/10/2026 (Anexo C)	Cuotas, suspensión, precios, versiones y soporte

0.2. Convenciones

Para separar lo documentado de lo propuesto se usan estas etiquetas:

Hecho documentado: aparece en [PS], [HU] o [EQ].

Decisión propuesta: recomendación de este documento; puede cambiarse con un ADR.

Supuesto: condición que se asume para avanzar; debe confirmarse.

Pendiente: decisión que no existe todavía; no se inventa su contenido.

Verificado en documentación oficial: dato comprobado en documentación oficial del proveedor en la fecha indicada; puede cambiar.

Los requisitos se citan con la numeración de [PS] (por ejemplo, RF03.1). Como [HU] usa la numeración plana anterior (RF01–RF34), cuando la cita proviene de [HU] se indica el identificador de la HU y su criterio (por ejemplo, HU20-CA04). La correspondencia completa está en el **Anexo A**.

1. Contexto y factores de decisión

1.1. Problema que resuelve el sistema

Hecho documentado: Un ciudadano necesita pedir ayuda urgente (SOS), reportar incidencias o compartir su ubicación de forma temporal y voluntaria. La central de la municipalidad (caso de estudio tipo B) debe recibir, verificar, asignar manualmente a un sereno y dar seguimiento a cada caso con trazabilidad, control de acceso por rol y protección de la privacidad ([PS] 1.2 y objetivo de producto).

1.2. Alcance real y exclusiones

Tabla 2. Alcance y exclusiones

Dentro del alcance (MVP)	Fuera del alcance
SOS con una sola activación, con o sin ubicación (RF03.1–RF03.3)	Canal oficial de emergencias reales ([PS] 1.4)
Reportes de incidencias con revisión, admisión, derivación o descarte (RF16, RF17)	Integración con PNP, Bomberos, SAMU o RENIEC ([PS] 1.4); la derivación es solo un registro (HU16-CA03)
Recepción en central, verificación, asignación manual y ciclo de atención (RF05–RF11)	Despacho automático, IA, optimización o cálculo de rutas ([PS] 1.4; PO en [PS] 2.1)
Historial, filtros, tiempos, mapa de casos y mapa de calor (RF05.2, RF12–RF14)	Rastreo continuo en segundo plano y recepción garantizada con la app cerrada ([PS] 1.5; RNF11)
Seguimiento del ciudadano y notificaciones dentro de la app (RF18)	Datos reales: todo es simulado ([PS] 1.4)
Acompañamiento de hasta 30 min, bienestar y avisos (RF20–RF23)	Telefonía integrada: el contacto se registra, no se realiza desde el sistema (RF08)
Administración de cuentas institucionales (RF15)	Aplicación para iOS (los dispositivos de prueba son Android y Windows, [EQ])

1.3. Actores y funcionalidades principales

Tabla 3. Actores, interfaces y funcionalidades

Actor	Interfaz	Funcionalidades principales	HU
Ciudadano	App móvil CAD (modo ciudadano)	Registro, SOS, reporte, mis casos, notificaciones, evaluación, acompañamiento, bienestar	HU01–HU02, HU06–HU10, HU30–HU32, HU34–HU37, HU41
Sereno	App móvil CAD (modo sereno)	Estado operativo y ubicación, recepción de despacho, desplazamiento, inicio de atención, cierre	HU18, HU21–HU24
Operador de central	Panel web de central	Sección urgente, verificación, clasificación, revisión de reportes, mapa operativo, asignación, historial, filtros, tiempos, acompañamientos, avisos	HU11–HU17, HU19–HU20, HU23–HU29, HU33, HU38–HU40, HU42–HU43
Administrador	Panel web de central	Crear, consultar, modificar y desactivar cuentas de operadores y serenos	HU04, HU05
Sistema (temporizador)	Servidor CAD	Vencimiento a los 30 min, avisos por reglas, eliminación por plazo (cuando DP22 se defina)	HU37, HU42, HU39-B

1.4. Equipo, plazo, presupuesto y entorno

Tabla 4. Factores de decisión del proyecto

Factor	Valor	Fuente	Tipo
Tamaño del equipo	3 integrantes con roles Frontend/UX, Backend/BD y QA/Seguridad/Documentación; trabajo multifuncional	[PS] 2.3	Hecho

Factor	Valor	Fuente	Tipo
Tecnología conocida	Backend en C#/.NET; sin preferencia para el frontend; el resto queda a criterio de este plan	[EQ]	Hecho
Estado actual	Sin código ni repositorio	[EQ]	Hecho
Plazo	Validación global 27–28/10/2026 y entrega objetivo 29/10/2026 ("fecha objetivo, no garantía")	[PS] 5	Hecho
Calendario	[PS] supone sprints desde el 07/09/2026; a 04/10/2026 no hay incremento: quedan 25 días naturales hasta la entrega objetivo	[PS] 4.5, [EQ]	Hecho (desfase)
Presupuesto	Capas gratuitas con o sin tarjeta; se acepta un plan mínimo que cumpla las restricciones	[EQ]	Hecho
Entorno de validación	Cloud gratuito o educativo ([PS] 1.5); se acepta un entorno controlado por ser un prototipo demostrativo	[PS], [EQ]	Hecho
Dispositivos	Android con Chrome; Windows con Chrome	[EQ]	Hecho
Navegadores del panel	Chrome y Firefox, versiones fijadas al iniciar la validación; 1366×768 y 1920×1080	RNF05	Hecho
Licencias	Software libre o capa gratuita, sin costos de licencia	[PS] 1.5	Hecho

1.5. Concurrencia, volumen y rendimiento

Tabla 5. Carga y rendimiento

Aspecto	Valor documentado	Comentario
Sesiones concurrentes	20 (ciudadanos, operadores y serenos), con al menos 5 acompañamientos activos durante 30 min (RNF03)	Es el único dato de concurrencia. No se diseña para más.
Rendimiento del SOS	≥95 % de al menos 100 SOS presentados y notificados en central en ≤3 s desde la aceptación del servidor (RNF02)	No es el tiempo de respuesta del sereno (HU28-CA04).
Frecuencia de posiciones	Acompañamiento: intento cada 15 s (HU35-CA01). Sereno: "intervalo configurado" (HU18-CA01), sin valor	Intervalo del sereno: parámetro configurable (posible DP15).
Volumen de datos	No definido; datos simulados	Estimación (no requisito): 5 sesiones × 4 puntos/min × 30 min ≈ 600 puntos por prueba de RNF03. Volumen trivial para PostgreSQL.
Disponibilidad	≥99 % en una prueba controlada de 8 h, sin presentarse como garantía municipal (RNF10)	99 % de 8 h = 4,8 min de interrupción máxima.

1.6. Seguridad, privacidad, consistencia y tiempo real

- **Seguridad:** hash de contraseñas con algoritmo específico y configuración documentada (RNF01); canal cifrado HTTPS y equivalente seguro para conexiones persistentes (RNF09); autorización por rol y rechazo de solicitudes directas (RF02, HU03-CA03).
- **Privacidad:** la ubicación del ciudadano solo se transmite en un SOS, en un reporte cuando él lo elige o durante un acompañamiento activo; el permiso del dispositivo no equivale a activación (RNF07). Notas internas y recorridos restringidos (RNF12). Conservación limitada de recorridos (RNF13, bloqueada por DP22).
- **Consistencia:** una asignación vigente por caso, una sesión activa por ciudadano, sin registros parciales ni duplicados por reiteración técnica (RNF08 y criterios de concurrencia de HU09, HU14–HU17, HU20, HU22–HU24, HU32, HU34, HU41).

- **Tiempo real:** SOS nuevos en central sin recargar (HU11-CA02), mapa y disponibilidad actualizados (HU19-CA03, HU25-CA05), notificación del despacho al sereno (HU21-CA01), notificaciones al ciudadano conservadas si estaba desconectado (HU31-CA02).

1.7. Demostración académica frente a operación real

Tabla 6. Lo que se valida frente a lo que no se promete

Aspecto	MVP académico (este plan)	Operación real (no se promete)
Datos	Usuarios y coordenadas simulados; ubicación simulada con *mock locations*	Datos personales reales, marco legal de protección de datos
Disponibilidad	≥99 % durante 8 h controladas, con el servidor sin suspensión	Servicio 24/7 con redundancia y guardias
Aplicación móvil	Android, app en primer plano y con conexión (condición validada de RNF11)	Seguimiento en segundo plano, notificaciones con la app cerrada, iOS
Notificaciones	Dentro de la app y del panel mientras están abiertos; recuperación al reconectar	Notificaciones push garantizadas
Escala	20 sesiones concurrentes	Cientos o miles de usuarios simultáneos
Seguridad	Controles de RNF01, RNF09 y RNF12 verificados en el entorno de validación	Auditoría, pentest, gestión de incidentes, copias de seguridad con retención

1.8. Discrepancias detectadas y tratamiento

Las discrepancias se registran sin modificar en silencio ningún requisito. Las que afectan a la arquitectura se resuelven con una decisión explícita o se dejan como pendientes.

Tabla 7. Discrepancias entre documentos y con las respuestas del equipo

ID	Discrepancia	Tratamiento en este plan
D1	Numeración RF distinta: [HU] usa RF01–RF34; [PS], RF01.1–RF23.2	Correspondencia por contenido en el Anexo A; se cita [PS].
D2	[HU] tiene HU20 y HU39 únicas; [PS] las divide en A/B como "propuesta de división"	Se respeta la división de [PS] solo para ordenar la construcción.
D3	HU11-CA01 (Sprint 1) y HU19-CA01 (Sprint 2) dependen del mapa de HU25 (Sprint 4)	Construir el componente de mapa operativo con HU11 y ampliarlo después.
D4	El stakeholder "Serenos" espera la "ruta exacta"; el PO excluye el ruteo; HU21 solo pide la ubicación	Sin ruteo. La app del sereno muestra el punto del caso.
D5	El nombre habla de despachar "al más cercano gracias al rastreo"; HU20-CA01 deja la comparación al operador	Sin cálculo de cercanía en el servidor; el mapa muestra caso y serenos.
D6	"Gestión de llamadas" ([PS] 1.4) frente al registro de intentos de contacto (RF08/HU12)	Sin telefonía; solo se muestra el teléfono y se registran intentos.
D7	"Suspensión tras 15 min y activación antes de cada demo" ([PS] 1.5) frente a RNF02 y RNF10	Plan sin suspensión (Render Starter) en la ventana de validación (ADR-05).
D8	La PostgreSQL gratuita de Render caduca a los 30 días [DOC]; el proyecto dura más	Base de datos en Neon Free; Render Postgres de pago como alternativa.
D9	HU41-CA03/CA04 solo consideran SOS "Pendiente" o "En atención"; omiten "Asignada" y "En camino"	Se implementa una consulta "SOS vigente" con la lista de estados como parámetro; la regla queda pendiente (posible DP14).
D10	Intervalo del sereno (HU18) y umbral de posición desactualizada (HU38) sin valor	Parámetros configurables; no se fijan valores de negocio.

ID	Discrepancia	Tratamiento en este plan
D11	[PS] 1.4/1.5 y RNF04 hablan de PWA y píxeles CSS; [EQ] pide una app móvil nativa	ADR-01: app .NET MAUI. Conviene actualizar [PS] 1.4, la limitación de plataforma y RNF04 (360/390/412 dp).
D12	RNF05 exige Chrome y Firefox; [EQ] menciona solo Chrome en Windows	Se mantiene RNF05 tal como está documentado; si el equipo excluye Firefox, debe modificar el RNF.
D13	Calendario de [PS] desde el 07/09; sin código al 04/10	Orden de construcción por riesgo (sección 11). Replanificar es decisión del PO.
D14	RF12.1 de [PS] cubre el historial de SOS y reportes, pero no el de sesiones de acompañamiento (HU39)	Se trata HU39 por contenido (Anexo A) y se recomienda ajustar RF12.1 o RF21.
D15	[PS] 1.6 menciona un "modelado de BD espacial"; ningún requisito exige consultas espaciales	Sin PostGIS (ADR-03); latitud y longitud como columnas numéricas con validación de rango.

1.9. Decisiones pendientes (DP01–DP22) y tratamiento arquitectónico

Hecho documentado: [PS] y [HU] citan DP01–DP22, pero ninguno de los dos documentos define su contenido, y [EQ] confirma que ninguna está resuelta. La columna "Tema" solo se rellena cuando el texto de las HU lo indica explícitamente; en los demás casos se marca como no inferible.

Tabla 8. Decisiones pendientes y su tratamiento

DP	Tema (según el texto de [HU]/[PS])	Tratamiento arquitectónico
DP01	Obligatoriedad del teléfono (HU12) y datos del registro (HU01)	Campo de teléfono en el perfil; obligatoriedad configurable en la validación.
DP02	Alta de administradores (HU02, HU04)	Semilla técnica de un administrador mediante variables de entorno, solo para validación (supuesto).
DP03	Cuenta desactivada con sesión abierta o asignación vigente (HU05-CA04)	Revocación mediante sello de seguridad ya disponible; no se aplica ninguna regla sobre la asignación hasta que se decida.
DP04	Alcance de "operador autorizado" (HU03, HU33, HU38, HU39)	Políticas de autorización con nombre propio; al inicio equivalen al rol Operador.
DP05	Visibilidad para el ciudadano (motivo de descarte, resultado, estado público) (HU17, HU24, HU30, HU31)	Un único mapeo "estado interno → estado público" y un DTO del ciudadano.
DP06	Datos del caso visibles para el sereno (HU21-CA05)	DTO del sereno mínimo; se amplía al decidir.
DP07	Catálogo de categorías (HU10, HU13, HU14, HU24, HU26)	Tabla de categorías con semilla; contenido mínimo de RF16.2.
DP09	Quién clasifica un SOS (HU13)	Política "ClasificarSos" asignada inicialmente al Operador.
DP10	Tratamiento de la referencia textual en el mapa (HU12, HU25)	La referencia se guarda separada de las coordenadas y no genera marcador.
DP16	Sereno sin geolocalización o que se desactiva con asignación vigente (HU18)	Transición no dibujada ni implementada hasta la decisión.
DP19	Origen y estado del caso urgente creado desde un aviso (HU43-CA05; [PS] S5-T09)	CA05 queda pendiente; se permite vincular con casos existentes.
DP20	Cierre sin conexión y solicitudes simultáneas de bienestar (HU36-CA03, HU40-CA04, HU42-CA05)	Cola local de cierre con reintento idempotente; el efecto en el panel queda pendiente.
DP22	Plazo de conservación de recorridos (HU39, RNF13)	Parámetro sin valor; tarea de eliminación deshabilitada; HU39-B bloqueada.

DP	Tema (según el texto de [HU]/[PS])	Tratamiento arquitectónico
DP08, DP11–DP15, DP17, DP18, DP21	No inferible con certeza a partir de los documentos	Se tratarán cuando se definan; ninguna exige, por su ubicación en las HU, cambiar la arquitectura (parámetros, validaciones o DTO).

2. Evaluación de alternativas

2.1. Categorías de decisión que no deben mezclarse

Tabla 9. Categorías de decisión arquitectónica

Categoría	Pregunta que responde	Opciones consideradas	Elección
Organización del despliegue	¿Cuántos procesos servidor se despliegan y operan?	Monolito · API + frontend separado · servicios	Monolito (un Servidor CAD)
Organización interna del código	¿Cómo se ordena el código dentro del servidor?	Capas horizontales · módulos por funcionalidad · ambos	Módulos por funcionalidad con capas ligeras dentro de cada uno
Patrón de presentación (panel)	¿Cómo se construye la interfaz web?	SPA (React/Angular) · Blazor · páginas del servidor con JS	Razor Pages (modelo de página) + módulos JS para mapa y tiempo real
Patrón de presentación (app)	¿Cómo se organiza la interfaz móvil?	MVVM · código detrás	MVVM con enlace de datos de MAUI

2.2. Comparación de alternativas de despliegue

Las tres alternativas comparten la app móvil MAUI y PostgreSQL; lo que cambia es cómo se organiza y despliega la parte servidor. La **A1** es la referencia centralizada sencilla.

Tabla 10. Comparación de alternativas de organización del despliegue

Criterio	A1 · Monolito modular: un Servidor CAD (API + SignalR + panel Razor + temporizador)	A2 · API ASP.NET Core + panel SPA separado (por ejemplo, React)	A3 · Servicios separados (casos, tiempo real, ubicaciones) con broker o backplane
Adecuación a requisitos	Cubre todos los RF/RNF. RNF02 se resuelve en proceso; RNF08 con transacciones locales.	Cubre todos los RF/RNF. Añade CORS o un proxy y gestión de tokens en el navegador.	Cubre los RF, pero RNF08 exige coordinar entre servicios (sagas o bloqueos distribuidos). Ningún requisito pide escalar componentes por separado.
Complejidad de implementación	Baja: un proyecto servidor, un lenguaje.	Media: dos bases de código web, dos pipelines, dos modelos de autenticación.	Alta: contratos entre servicios, mensajería, consistencia eventual.
Despliegue y operación	Un servicio web y una base de datos.	Dos servicios web (o un sitio estático más la API) y una base de datos.	Tres o más servicios, broker o Redis y una o varias bases de datos.
Facilidad de pruebas	Alta: pruebas de integración en proceso con PostgreSQL real.	Media: se necesitan pruebas E2E del SPA contra la API.	Baja: entornos con varios servicios y pruebas de contrato.
Ajuste al equipo (C#/.NET)	Total: C# en servidor y vistas; JS solo para mapa y tiempo real.	Parcial: hay que aprender un framework SPA y su ecosistema.	Bajo: requiere experiencia en sistemas distribuidos.
Costos y limitaciones	1 servicio web: Free o Starter (USD 7/mes) [DOC].	2 servicios o un sitio estático adicional; más tiempo de compilación.	Varios servicios de pago para evitar la suspensión; los planes gratuitos se suspenden por separado.
Riesgos	Un fallo del proceso detiene todo (aceptable en un MVP). Riesgo de acoplamiento si no se respetan los módulos.	Desalineación entre API y SPA; más superficie de seguridad (tokens en el navegador).	Sobreingeniería; no llegar a la fecha; fallos parciales difíciles de diagnosticar.
Decisión	Elegida. Es la opción más sencilla que cumple RNF02, RNF03 y RNF08 con el stack que conoce el equipo.	Descartada: su beneficio (independizar el frontend) no responde a ningún requisito.	Descartada: ninguna necesidad verificada la justifica (ver sección 12).

2.3. Recomendación

Decisión propuesta: **A1 · Monolito modular**, con el Servidor CAD como única unidad desplegable de servidor, el panel servido por el mismo proceso, PostgreSQL como fuente de verdad y la app móvil MAUI como cliente. La separación entre módulos se mantiene **dentro del código**, no mediante procesos distintos.

3. Arquitectura seleccionada

3.1. Nombre y descripción

Monolito modular con capas ligeras y cliente móvil nativo. Un único proceso ASP.NET Core 10, el **Servidor CAD**, concentra:

- la **API REST** que usan la app móvil y el JavaScript del panel;
- el **Hub SignalR**, que solo envía eventos de tiempo real (y recibe acuses de presentación para medir RNF02);
- las **páginas Razor** del panel de central y de administración;
- el **temporizador de dominio** (*BackgroundService*), que vence sesiones, evalúa reglas de avisos y, cuando DP22 se defina, elimina puntos vencidos.

Toda escritura pasa por un caso de uso del módulo correspondiente, se confirma en **PostgreSQL** dentro de una transacción y solo después se publica como evento. Si un cliente pierde eventos, recupera el estado consultando la base de datos.

3.2. Unidades desplegables

Tabla 11. Unidades desplegables y almacenes

Unidad	Tecnología	Dónde se ejecuta	Contiene
Servidor CAD	ASP.NET Core 10 en contenedor Docker	Render Web Service (Free en desarrollo, Starter en validación)	API, Hub, panel (HTML/JS servido por el mismo proceso), temporizador
Base de datos CAD	PostgreSQL gestionado	Neon (plan Free); alternativa Render Postgres de pago	Todos los datos persistentes
App móvil CAD	.NET MAUI 10, Android (APK)	Teléfonos Android del equipo y de los usuarios de prueba	Modo ciudadano y modo sereno, según el rol tras iniciar sesión

Decisión propuesta: El **panel no es una unidad desplegable separada**: es un contenedor C4 distinto porque se ejecuta en el navegador, pero el Servidor CAD lo entrega. **Una sola app** con dos modos evita mantener dos APK; la separación ciudadano/sereno se garantiza en el servidor, no por tener aplicaciones distintas.

3.3. Módulos funcionales internos

Un **módulo** es una carpeta del Servidor CAD con su dominio, sus casos de uso y sus endpoints. **No** es un servicio desplegable. No se crea un servicio, repositorio ni interfaz por entidad.

Tabla 12. Módulos del Servidor CAD

Módulo	Responsabilidades	Entidades propias	HU principales
Cuentas	Registro de ciudadano, inicio de sesión (cookie y JWT), cuentas institucionales, desactivación, políticas de rol	Usuario (Identity), PerfilCiudadano	HU01–HU05
Casos	SOS (emisión, ubicación, cancelación), reportes (registro, revisión, admisión, derivación, descarte), verificación, clasificación, historial, notificaciones al ciudadano, evaluación	Caso, EventoCaso, Categoria, Notificacion, Evaluacion	HU07–HU17, HU27, HU30–HU32
Despacho	Estado operativo y última posición del sereno, disponibilidad derivada, asignación, desplazamiento, inicio de atención y cierre	EstadoSereno, Asignacion	HU18–HU24
Acompañamiento	Sesiones de 30 min, puntos, bienestar, avisos, retención	SesionAcompanamiento, PuntoAcompanamiento, SolicitudBienestar, Aviso, EventoAviso	HU34–HU43

Módulo	Responsabilidades	Entidades propias	HU principales
Consultas	Lecturas para el panel: estado inicial y resincronización, filtros, tiempos, mapa de calor, evaluaciones	Ninguna (solo lectura)	HU25, HU26, HU28, HU29, HU33

3.4. Capas

Dentro de cada módulo hay tres capas lógicas (presentación, aplicación y dominio) y una infraestructura compartida. Se describen con detalle en la sección 6. No se añaden capas de repositorio, *mappers* automáticos ni mediadores, porque EF Core ya ofrece la unidad de trabajo y las consultas.

3.5. Dependencias permitidas y prohibidas

Tabla 13. Reglas de dependencia

Desde	Puede depender de	No puede depender de
Presentación (endpoints, páginas, hub, temporizador)	Aplicación del mismo u otro módulo; configuración de Identity/JWT	CadDbContext para escribir; reglas de negocio propias
Aplicación de un módulo	Su dominio; CadDbContext; INotificador; la aplicación pública de otro módulo (por ejemplo, Acompañamiento → Casos.EmitirSos)	Endpoints, Hub o tipos HTTP; entidades internas de otro módulo para modificarlas
Dominio	Nada externo (solo .NET base)	EF Core, ASP.NET Core, SignalR, otros módulos
Consultas	CadDbContext en modo solo lectura (*AsNoTracking*) y proyecciones a DTO	Escrituras de cualquier tipo
App móvil	Proyecto Cad.Contratos (DTO y nombres de eventos)	Código del servidor

3.6. Ubicación de reglas, validaciones y autorización

Tabla 14. Dónde vive cada tipo de regla

Tipo de regla	Dónde se implementa	Ejemplo
Validación de formato	Endpoint (anotaciones o validación manual del DTO)	Puntuación 1–5 (HU32-CA02); campos obligatorios del reporte (HU10-CA02)
Validación en el cliente	App y panel, solo para usabilidad; el servidor revalida siempre	Botón deshabilitado sin motivo de descarte
Autorización por rol	Políticas de ASP.NET Core en cada endpoint y en el Hub	Solo el Operador asigna (HU20-CA09)
Autorización por titularidad	Caso de uso (aplicación) antes de modificar	Solo el sereno asignado confirma (HU22-CA03); el ciudadano solo ve sus casos (HU30-CA04)
Reglas de estado e invariantes	Dominio (métodos de la entidad que validan la transición)	"Asignar" solo desde "Pendiente" (HU20-CA02)
Unicidad y concurrencia	Base de datos (índices únicos, token xmin), con la traducción del conflicto a 409 en la aplicación	Una asignación vigente por caso y por sereno (HU20-CA04/05)
Reglas temporales	Dominio (cálculo con vence_en) + temporizador + comprobación al acceder	Vencimiento a los 30 min (HU37-CA01)
Visibilidad de datos	DTO por rol en la aplicación; nunca se serializa la entidad	Notas internas fuera del DTO del ciudadano (HU24-CA04)

3.7. Acceso a datos y límites transaccionales

Cada caso de uso de escritura usa **una sola transacción**: carga, valida en el dominio, modifica y llama a `SaveChanges` una vez. Los eventos de historial y las notificaciones persistidas se guardan en la misma transacción que el cambio de estado. La publicación por SignalR se hace **después** de confirmar la transacción. El detalle por operación está en la sección 9.3.

3.8. Comunicación entre componentes

Tabla 15. Canales de comunicación

Origen → destino	Mecanismo	Uso	Autenticación
App → Servidor	HTTPS + JSON (REST)	Todas las escrituras y consultas de la app	JWT Bearer
Servidor → App	SignalR sobre WSS	Despacho asignado, cambios de caso, avances, cierre de sesión, solicitud de bienestar	JWT (en la conexión)
Navegador → Servidor	HTTPS (páginas Razor) + JSON (fetch)	Pantallas y acciones del operador y del administrador	Cookie HttpOnly + token antifalsificación
Servidor → Panel	SignalR sobre WSS	SOS nuevo, cambios de caso, posiciones, disponibilidad, avisos	Cookie
Panel → Servidor (Hub)	SignalR	Solo acuse "Presentado" para medir RNF02	Cookie
Servidor → PostgreSQL	Npgsql sobre TLS	Persistencia	Usuario y contraseña en variable de entorno
App / Panel → OpenStreetMap	HTTPS	Teselas de mapa	Ninguna; User-Agent propio en la app

Tabla 16. Endpoints principales (propuesta inicial)

Endpoint	Rol	HU
POST /api/cuentas/registro · POST /api/auth/token	Anónimo	HU01, HU02
POST /api/sos · POST /api/sos/{id}/cancelacion	Ciudadano	HU07, HU08, HU09
POST /api/reportes · GET /api/mis-casos · GET /api/notificaciones	Ciudadano	HU10, HU30, HU31
POST /api/acompanamientos · POST /api/acompanamientos/{id}/puntos · POST /api/acompanamientos/{id}/cierre	Ciudadano	HU34–HU36
POST /api/bienestar/{id}/presentacion · /respuesta	Ciudadano	HU41
PUT /api/sereno/estado-operativo · POST /api/sereno/posicion · GET /api/sereno/asignacion-vigente	Sereno	HU18, HU21
POST /api/casos/{id}/desplazamiento · /inicio-atencion · /cierre	Sereno asignado u Operador (según HU)	HU22–HU24
GET /api/central/estado (resincronización) · GET /api/casos?filtros · GET /api/casos/{id}/historial	Operador	HU11, HU19, HU25–HU27
POST /api/casos/{id}/contactos · /clasificacion · /revision · /admision · /derivacion · /descarte · /asignacion	Operador	HU12–HU17, HU20
GET /api/acompanamientos/activos · POST /api/acompanamientos/{id}/bienestar · /api/avisos/*	Operador	HU38–HU40, HU42, HU43
/panel/admin/cuentas (Razor Pages)	Administrador	HU04, HU05

3.9. Infraestructura compartida y separación real

- **Comparten proceso y código:** todos los módulos, la API, el Hub, el panel y el temporizador. Comparten también CadDbContext y la configuración de autenticación.
- **Comparten contratos:** el proyecto `Cad.Contratos` (DTO, nombres de eventos y enumeraciones públicas) lo usan el servidor y la app.
- **Necesitan separación real:** el **proceso servidor** y la **base de datos**, porque la persistencia debe sobrevivir a reinicios y despliegues; la **app móvil**, porque se ejecuta en otro dispositivo; y los **secretos**, que van en variables de entorno y nunca en el repositorio.
- **Separación lógica obligatoria aunque compartan proceso:** las notas internas frente a los datos públicos (RNF12), y los recorridos de acompañamiento frente al historial de casos (HU27-CA05, HU39-CA01).

4. Stack tecnológico

Un stack principal coherente, con C#/.NET como eje. Las versiones se fijan a la línea con soporte vigente; los parches exactos se toman de NuGet y npm al crear el repositorio.

Tabla 17. Stack tecnológico seleccionado

Área	Tecnología	Función	Requisito o restricción	Frente a una alternativa plausible	Costos, limitaciones y riesgos
Backend	ASP.NET Core 10 (LTS), Minimal APIs	API REST, composición del servidor	[EQ] C#/.NET; RNF14	.NET 8 LTS termina soporte el 10/11/2026 [DOC]; .NET 10 tiene soporte hasta el 14/11/2028	Requiere SDK 10 en todos los equipos
Tiempo real	ASP.NET Core SignalR (WebSocket con alternativas automáticas)	Eventos a panel y app; grupos por rol e identidad	RNF02, HU11-CA02, HU19-CA03, HU21-CA01	WebSocket propio: habría que reimplementar reconexión, grupos y autenticación	Sin cola persistente: un evento perdido se recupera consultando la base de datos
Panel web	Razor Pages + JavaScript (módulos ES) + Leaflet 1.9.4 + cliente JS de SignalR	Sección urgente, mapa operativo, formularios	RNF05, RF05.1, RF05.2	SPA en React: segundo proyecto y otro lenguaje sin requisito que lo exija; Blazor: interop JS para Leaflet	Política de reproducción automática del navegador: el sonido se habilita con un clic de "Iniciar turno" (por validar)
App móvil	.NET MAUI 10 (Android), MVVM con enlaces de MAUI	Modos ciudadano y sereno	[EQ] app nativa; RNF04; RNF06	Kotlin nativo: otro lenguaje; Flutter: Dart y sin contratos compartidos	MAUI 10 con soporte hasta el 11/05/2027 [DOC]. Compilación Android lenta; probar pronto en dispositivo (SP-A, SP-F)
Mapas	Leaflet 1.9.4 (panel), Mapsui.Maui 5.x (app), teselas de OpenStreetMap	Mapa operativo, selección de punto en reporte, ubicación del caso	RF05.2, HU10-CA03, HU19, HU25; [PS] 1.4 (OSM/Leaflet)	Google Maps: clave, cuenta de facturación y otro proveedor	Política de teselas OSM: atribución visible, User-Agent propio, sin descarga masiva ni SLA [DOC]
Mapa de calor	Plugin Leaflet.heat (opcional)	Mapa de concentración	RF14 / HU29 (Could)	Cálculo en el servidor: innecesario con este volumen	Plugin antiguo (por validar con Leaflet 1.9.4); alternativa: círculos con intensidad
Geolocalización	API Geolocation de MAUI (Microsoft.Maui.Devices.Sensors)	Posición del SOS, reporte, acompañamiento, sereno	RF03.2, RF04, RF06, RF21; RNF07	Servicio en primer plano de Android: no requerido y contradice RNF11	Solo con la app en primer plano (condición validada)
Base de datos	PostgreSQL gestionado (Neon Free; versión mayor ofrecida por el proveedor)	Fuente de verdad	RNF08 (índices únicos parciales, transacciones)	MySQL: sin índices parciales; SQLite: concurrencia y despliegue limitados	Neon Free: 100 CU-h/mes, escala a cero tras 5 min, 1 GB [DOC]; el arranque en frío debe medirse (SP-D)
Acceso a datos	EF Core 10 + Npgsql.EntityFrameworkCore.PostgreSQL 10.x; migraciones de EF Core	Mapeo, transacciones, token xmin	RNF08, RNF14	Dapper: más SQL manual y migraciones aparte	Las migraciones se aplican al arrancar (una sola instancia)
Autenticación	ASP.NET Core Identity (PBKDF2-HMAC-SHA512, 100 000 iteraciones por defecto, formato V3) + cookie (panel) + JWT Bearer (app)	Credenciales, roles, sesiones, revocación	RNF01, RF01, RF02, RNF09	Proveedor externo (Auth0/Entra): dependencia y configuración sin necesidad	Dos esquemas de autenticación; JWT sin token de refresco (vida configurable)

Área	Tecnología	Función	Requisito o restricción	Frente a una alternativa plausible	Costos, limitaciones y riesgos
Pruebas	xUnit + WebApplicationFactory + PostgreSQL real (Docker Compose local, contenedor de servicio en CI)	Unitarias de dominio; integración de concurrencia, idempotencia y autorización	RNF08, RNF12, HU03-CA04	Proveedor InMemory de EF: no reproduce índices ni concurrencia	Requiere Docker Desktop en los equipos
Carga y disponibilidad	Consola .NET propia (Cad.PruebaCarga) con HttpClient y cliente SignalR	RNF02, RNF03 y monitor de RNF10	RNF02, RNF03, RNF10	k6/JMeter: soporte del protocolo SignalR limitado o adicional	Hay que escribirla, aunque reutiliza Cad.Contratos
Despliegue	Docker (imagen aspnet:10.0) en Render Web Service; APK firmado e instalado manualmente	Publicación con HTTPS y WSS	RNF09, RNF10; presupuesto mínimo	Azure App Service: más configuración; VPS: operación propia	Free: suspensión tras 15 min y arranque de ~1 min [DOC]; Starter: USD 7/mes prorrateado
Versiones y CI	GitHub (ramas + pull requests) + GitHub Actions (compilar y probar)	RNF14; despliegue automático desde main	RNF14	Sin CI: errores tardíos	Minutos gratuitos de Actions según la cuenta (por validar)
Registros y monitoreo	ILogger con salida JSON a consola (logs de Render) + health checks (/salud, /salud/lista) + monitor de Cad.PruebaCarga	Diagnóstico y medición de RNF10	RNF10; prohibiciones de registro de RNF01/RNF07	Plataforma de observabilidad (Seq, Grafana): no la exige ningún requisito	Los logs de Render tienen retención limitada (por validar); se exportan los informes de prueba

4.1. Verificaciones de viabilidad (documentación oficial, 04/10/2026)

Tabla 18. Hechos verificados que condicionan la viabilidad

Hecho verificado	Impacto en el diseño
Render Free suspende el servicio tras 15 min sin tráfico entrante; los mensajes WebSocket cuentan como tráfico; arrancar de nuevo tarda unos 1 min; 750 h gratuitas al mes; puede reiniciarse en cualquier momento.	Sirve para desarrollo, pero no para las pruebas de RNF02 ni RNF10. Durante la validación se usa Starter (ADR-05).
La PostgreSQL gratuita de Render caduca 30 días después de crearse (con 14 días de gracia) y no tiene copias de seguridad.	Se descarta para este proyecto (D8).
Render: Starter USD 7/mes; Postgres de pago desde USD 6/mes; facturación prorrateada por segundo; espacio de trabajo Hobby gratuito.	Costo máximo de USD 13/mes si se pagaran ambos un mes completo.
Neon Free: 1 GB por proyecto, 100 CU-h/mes por proyecto, escala a cero tras 5 min (no se puede desactivar); si se agota el cómputo, el proyecto se suspende hasta el siguiente periodo.	El temporizador no consulta la base de datos si no hay sesiones o avisos activos; /salud no toca la base de datos. Alternativa: Render Postgres.
.NET 10 LTS con soporte hasta el 14/11/2028; .NET 8 y .NET 9 terminan el 10/11/2026; MAUI 10 con soporte hasta el 11/05/2027; MAUI 9 ya sin soporte.	Se usa .NET 10 en todos los proyectos.
Mapsui.Maui 5.1.0 (MIT) es compatible con net10.0-android.	Mapas OSM en la app sin clave.
Política de teselas de OSM: atribución, User-Agent identificable, caché, sin precarga ni descarga masiva, sin SLA.	Volumen de prueba bajo; User-Agent "CAD-MVP-USMP/1.0" en la app.
Identity (formato V3): PBKDF2 con HMAC-SHA512, sal de 128 bits, subclave de 256 bits, 100 000 iteraciones por defecto.	Configuración documentable para RNF01.
Cloudflare Quick Tunnels: solo para pruebas, límite de 200 solicitudes en curso, sin SSE; el soporte de WebSocket no está documentado.	Solo como contingencia del entorno controlado; WebSocket pendiente de validar.

4.2. Pendientes de validación técnica

- Disponibilidad de la región de EE. UU. Este en Render y AWS us-east-1 en Neon al crear los servicios (se recomienda la misma región para reducir latencia).
- Tiempo de arranque en frío de Neon tras 5 min sin uso y su efecto en el primer SOS de una demostración (SP-D).
- Comportamiento de la geolocalización, los permisos y la reconexión de SignalR en los teléfonos Android reales del equipo (SP-A, SP-B).
- Política de reproducción automática de sonido en Chrome y Firefox para el aviso de HU11 (SP-E).
- Compatibilidad de Leaflet.heat con Leaflet 1.9.4 (solo si se implementa HU29).

4.3. Costo estimado

Precios tomados de render.com/pricing el 04/10/2026; pueden cambiar. No se asume ninguna garantía de servicio de las capas gratuitas.

Tabla 19. Costo estimado con el presupuesto mínimo

Periodo	Configuración	Costo
Desarrollo	Render Web Service Free + Neon Free + GitHub	USD 0
Validación (27–28/10) y demostraciones	Cambiar el servicio a Starter solo durante esos días (prorrateo por segundo)	Fracción de USD 7; como máximo USD 7 por un mes completo
Contingencia de base de datos	Render Postgres de pago si Neon agota cuota o el arranque en frío afecta a RNF02	Hasta USD 6/mes, prorrateado

Periodo	Configuración	Costo
Entorno controlado (alternativa aceptada por [EQ])	Docker Compose en un PC del equipo + túnel HTTPS	USD 0, pero depende de ese PC y de la red local; menos representativo

5. Diagramas de procesos

Se representan los **procesos reales** del sistema (SOS, reporte, acompañamiento, bienestar y avisos), no cada operación CRUD. Son diagramas de flujo Mermaid agrupados por actor; **no son BPMN formal**. Como [EQ] no exige ninguna notación concreta, no se añade una especificación BPMN/Bizagi. Convenciones: rombo = decisión; óvalo = inicio o fin; flecha punteada = condición excepcional o efecto diferido; los números ① ② ③ ④ indican el orden en que interviene cada actor.

5.1. Proceso SOS: emisión y registro

Objetivo	Actores	Requisitos e historias
Registrar un SOS con una sola activación, con o sin ubicación, sin duplicados y confirmando solo tras la respuesta del servidor	Ciudadano, Servidor CAD	RF03.1, RF03.2, RNF06, RNF07, RNF08; HU07 (CA01–CA07), HU08 (CA01–CA05)

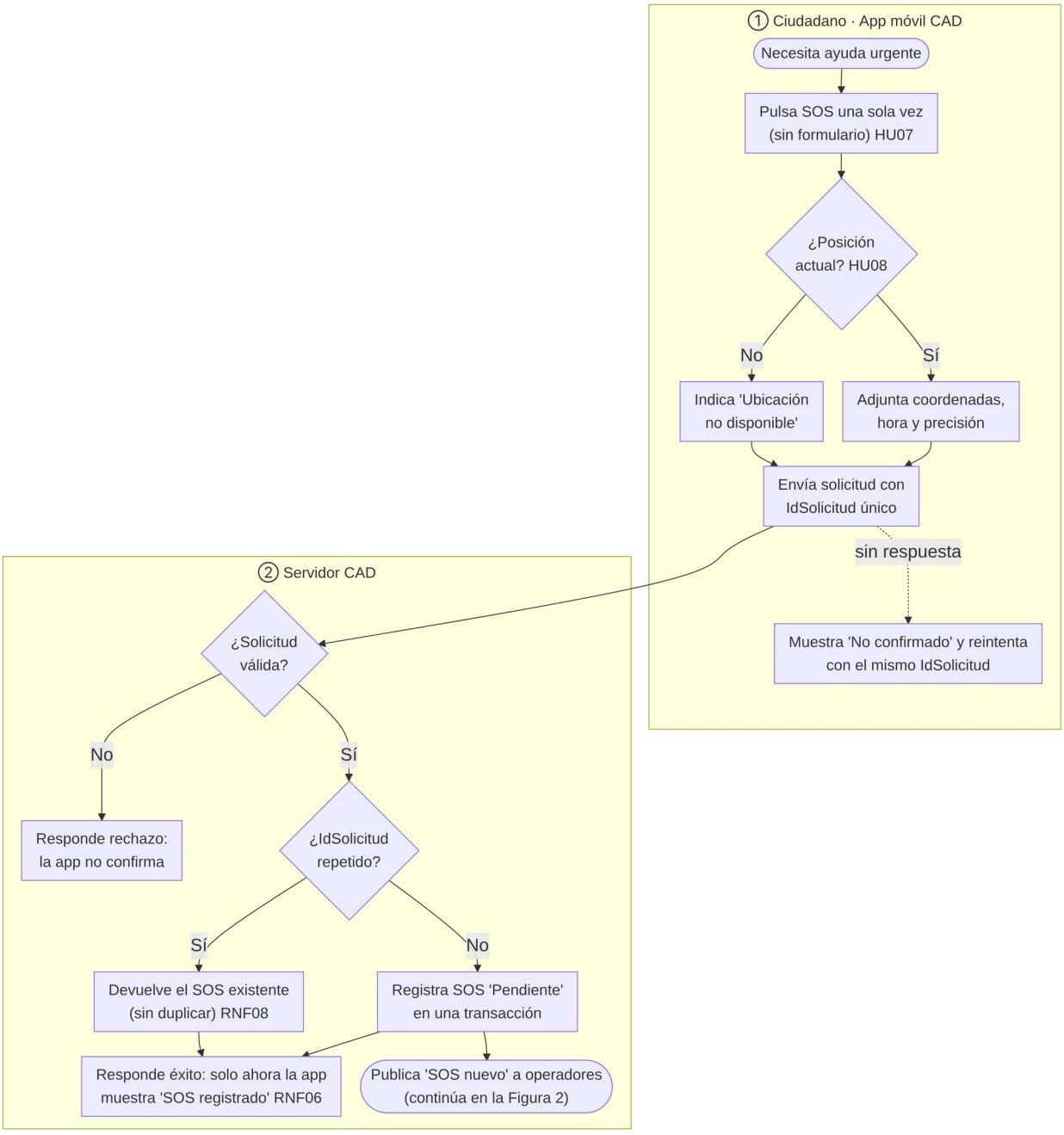


Figura 1. Proceso SOS · emisión y registro (Mermaid)

Cómo leerlo. El ciudadano (①) pulsa SOS una vez. La app intenta obtener la posición **actual**; si no la consigue, envía la alerta igualmente indicando "Ubicación no disponible" (HU08-CA02), sin usar una posición antigua (HU08-CA03). Cada SOS lleva un **IdSolicitud** generado en el teléfono. El servidor (②) valida la autenticación, el rol y los datos; si el IdSolicitud ya existe, devuelve el SOS registrado en lugar de crear otro (HU07-CA07). Solo cuando la respuesta llega, la app muestra "SOS registrado" (HU07-CA03). Si no llega, reintenta con el mismo IdSolicitud. La publicación a la central ocurre **después** de confirmar la transacción y continúa en la Figura 2.

5.2. Proceso SOS: recepción, despacho y atención

Objetivo	Actores	Requisitos e historias
Presentar el SOS en central, asignarlo manualmente a un sereno disponible y completar la atención	Operador, Sereno, Servidor CAD	RF05.1, RF07–RF11; RNF02, RNF08; HU11, HU12 (CA06), HU13 (CA02), HU19, HU20 (CA01–CA06), HU21–HU24

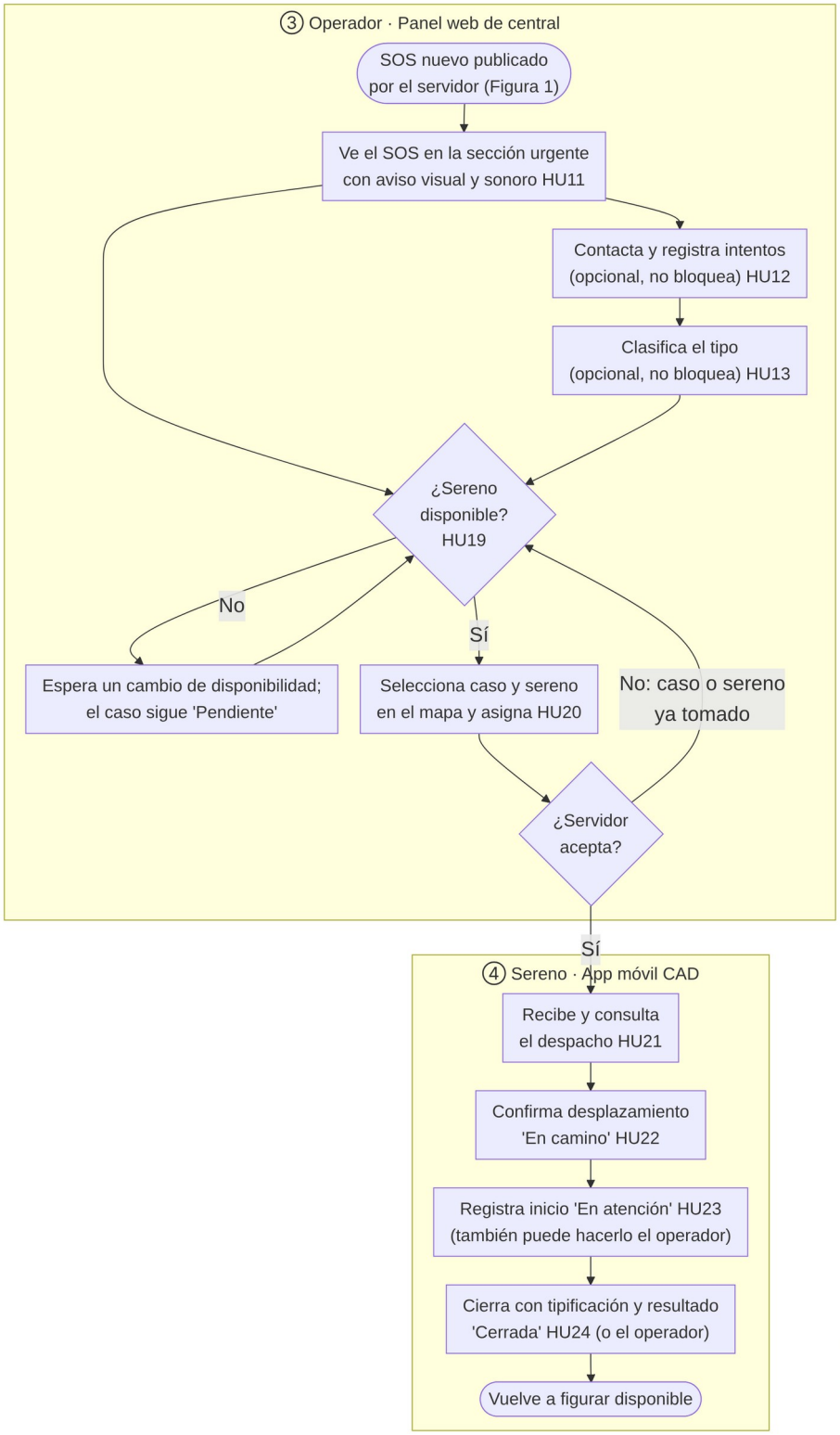


Figura 2. Proceso SOS · recepción, despacho y atención (Mermaid)

Cómo leerlo. La verificación (HU12) y la clasificación (HU13) son **opcionales** y no bloquean la asignación (HU12-CA06, HU13-CA02), por eso existe una flecha directa de la sección urgente a la decisión "¿Sereno disponible?". Si dos operadores intentan asignar a la vez, el servidor acepta solo una asignación y el otro operador vuelve a elegir (HU20-CA04/05). El sereno asignado avanza el caso por "En camino", "En atención" y "Cerrada"; el operador también puede registrar el inicio y el cierre (HU23-CA02, HU24-CA05). Al cerrar, el sereno vuelve a figurar disponible (HU24-CA07). La aceptación o el rechazo del despacho por el sereno **no** están definidos (HU20, alcance) y no se dibujan.

5.3. Proceso de reporte de incidencias

Objetivo	Actores	Requisitos e historias
Registrar un reporte y decidir su tratamiento sin confundirlo con un SOS	Ciudadano, Operador, Servidor CAD	RF16.1, RF16.2, RF17, RF18; RNF07, RNF08; HU10, HU14–HU17, HU20 (CA07), HU30, HU31

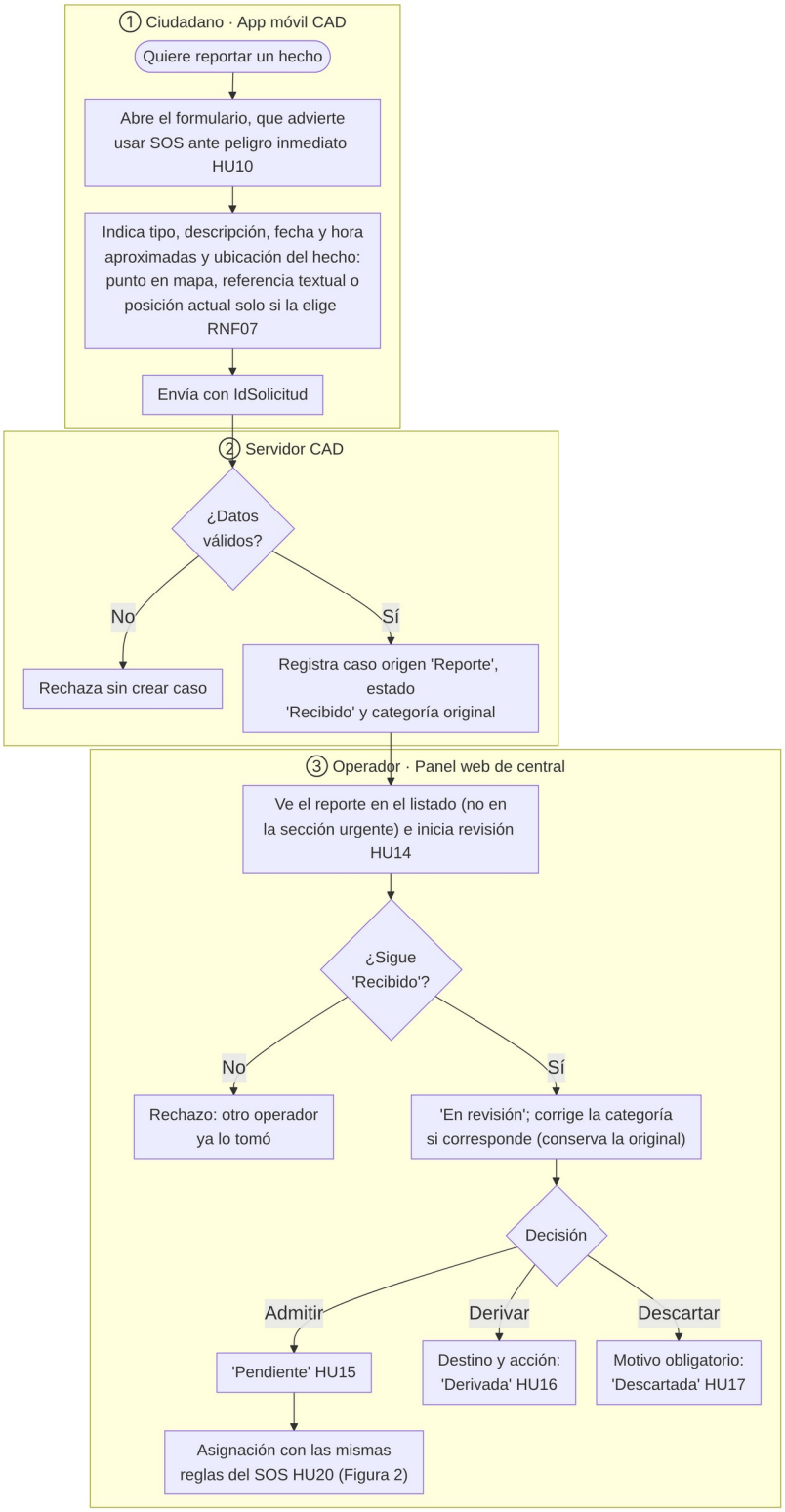


Figura 3. Proceso de reporte de incidencias (Mermaid)

Cómo leerlo. El formulario advierte que ante peligro inmediato debe usarse SOS (HU10-CA05). La ubicación del hecho puede indicarse con un punto en el mapa, una referencia textual o la posición actual, y esta última **solo si**

el ciudadano la elige (HU10-CA04, RNF07). El reporte entra como "Recibido" en el listado, no en la sección urgente. La revisión controla la concurrencia: si dos operadores la inician a la vez, solo uno queda como responsable (HU14-CA05). La decisión tiene tres salidas excluyentes; "Admitir" deja el caso "Pendiente" y la asignación sigue las mismas reglas que el SOS (Figura 2). Qué ve el ciudadano de cada cambio depende de **DP05**.

5.4. Proceso de acompañamiento temporal: activación y transmisión

Objetivo	Actores	Requisitos e historias
Compartir voluntariamente la ubicación durante un máximo de 30 min, con una sola sesión activa	Ciudadano, Operador, Servidor CAD	RF20.1, RF21; RNF07, RNF08, RNF11; HU34 (CA01–CA08), HU35 (CA01–CA05), HU38

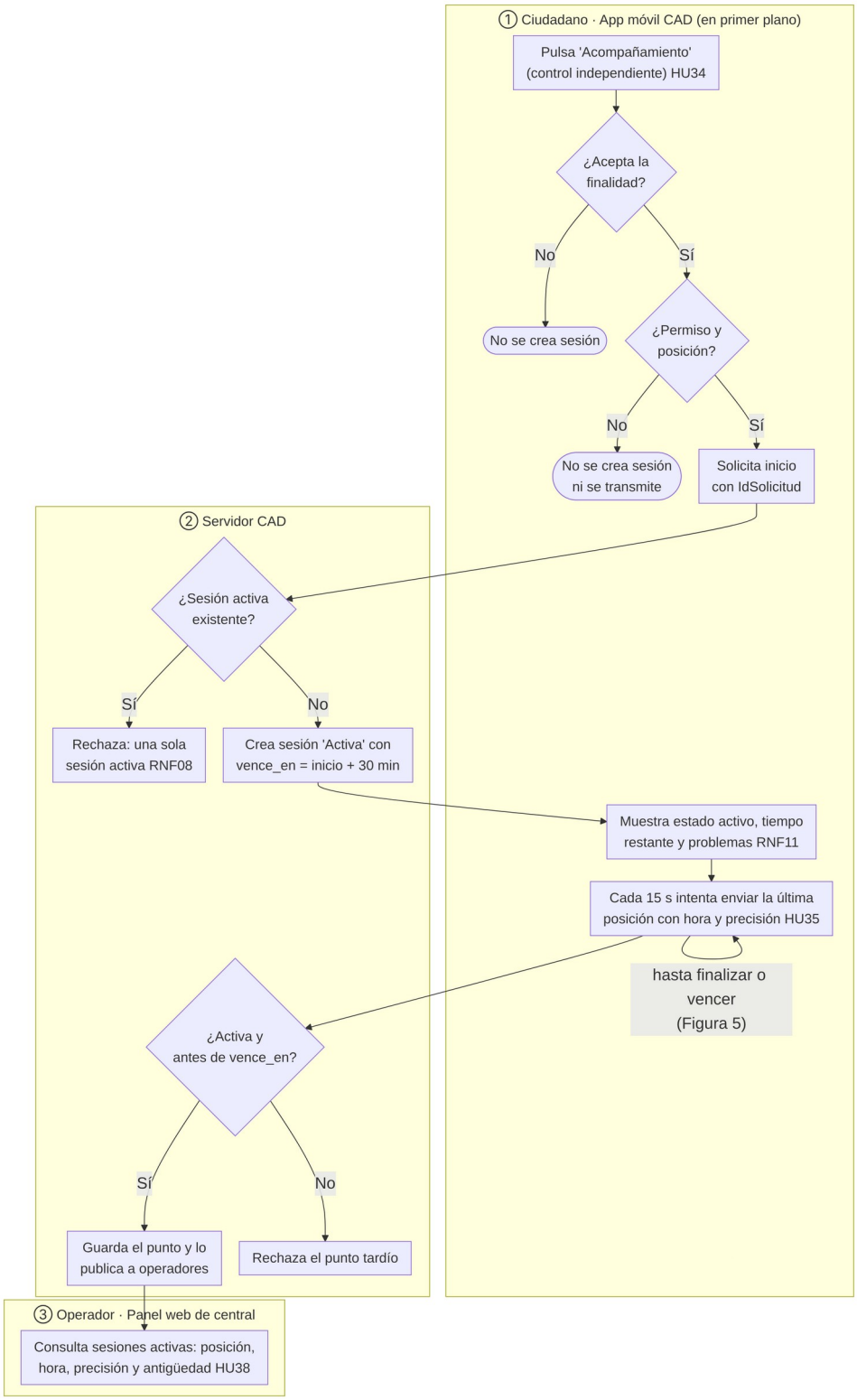


Figura 4. Acompañamiento · activación y transmisión (Mermaid)

Cómo leerlo. La sesión solo se crea si el ciudadano acepta la finalidad, tiene permiso y hay posición disponible (HU34-CA02/CA03). El servidor registra el inicio y `vence_en` (inicio + 30 min) y rechaza una segunda sesión activa (HU34-CA06). La app intenta enviar la última posición cada 15 s **mientras está en primer plano** (condición validada, RNF11). El servidor solo guarda puntos de una sesión activa y no vencida. El operador ve posición, hora de obtención, precisión y antigüedad (HU38).

5.5. Proceso de acompañamiento temporal: finalización y vencimiento

Objetivo	Actores	Requisitos e historias
Garantizar que la transmisión termina cuando el ciudadano decide o a los 30 min, incluso sin conexión	Ciudadano, Servidor CAD	RF20.2; RNF07, RNF08, RNF11; HU36 (CA01–CA07), HU37 (CA01–CA06); DP20

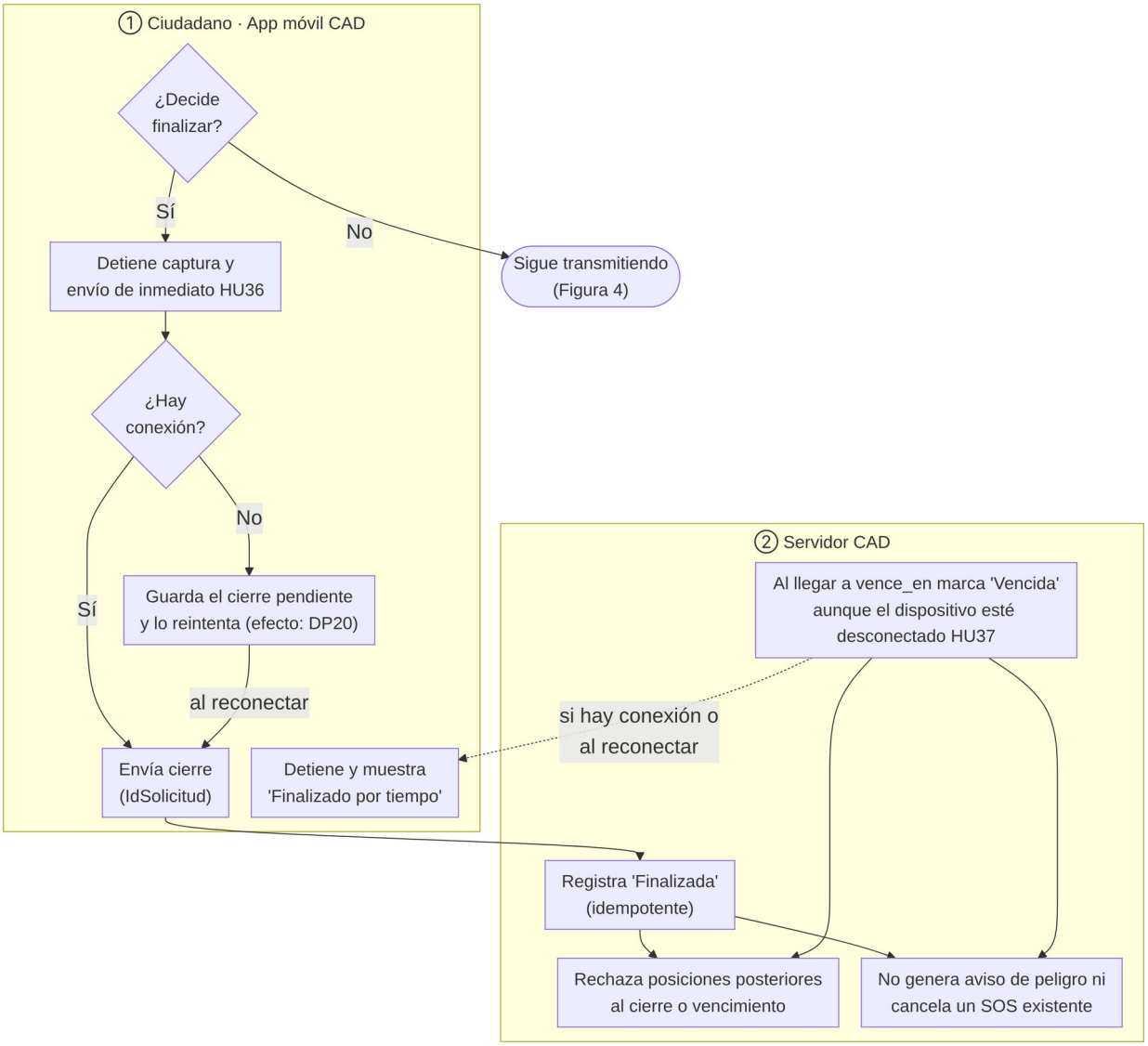


Figura 5. Acompañamiento · finalización y vencimiento (Mermaid)

Cómo leerlo. Al finalizar, la app **detiene la captura de inmediato**, aunque no haya conexión (HU36-CA01/CA03). El cierre se guarda localmente y se reintenta con el mismo IdSolicitud. Qué ocurre en el panel mientras el cierre no llega es **DP20 (pendiente)**. El vencimiento lo decide el servidor con `vence_en` y no depende del teléfono (HU37-CA01). Los puntos que llegan tarde se rechazan (HU36-CA04, HU37-CA04). Ni la finalización ni el vencimiento se interpretan como peligro ni cancelan un SOS (HU36-CA05/CA06, HU37-CA05).

5.6. Proceso de confirmación de bienestar

Objetivo	Actores	Requisitos e historias
Permitir que la central consulte el estado del ciudadano durante un acompañamiento sin generar despachos automáticos	Operador, Ciudadano, Servidor CAD	RF22; RNF08; HU40 (CA01–CA04), HU41 (CA01–CA07); DP14, DP20

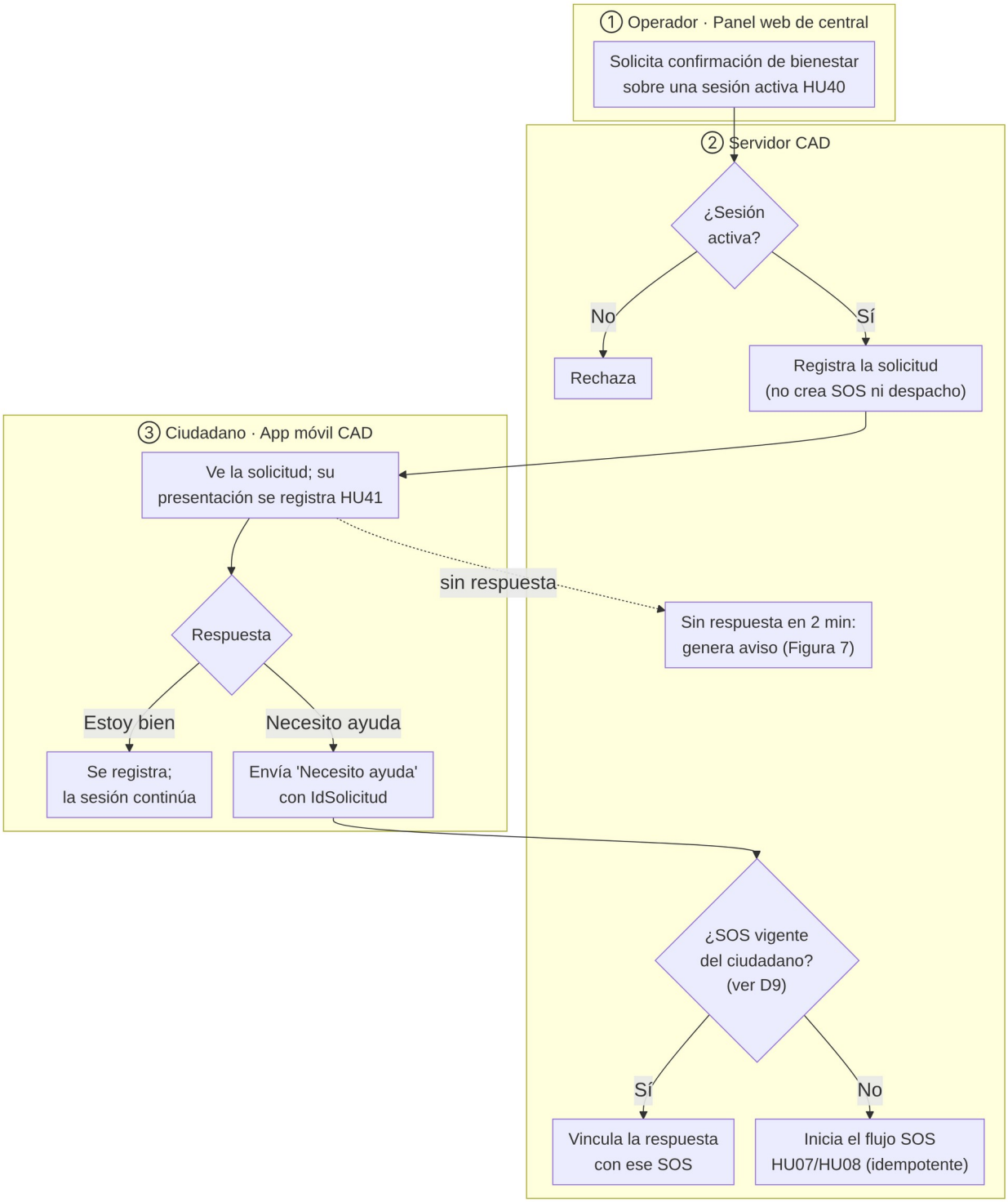


Figura 6. Confirmación de bienestar (Mermaid)

Cómo leerlo. La solicitud solo es válida sobre una sesión activa y no crea SOS ni despacho (HU40-CA02/CA03). "Necesito ayuda" inicia el flujo SOS, o lo vincula con un SOS vigente sin duplicarlo (HU41-CA03/CA04). Qué estados cuentan como "SOS vigente" está **pendiente** (discrepancia D9). Una solicitud presentada y sin respuesta durante 2 min genera un aviso (Figura 7).

5.7. Proceso de avisos del acompañamiento

Objetivo	Actores	Requisitos e historias
Alertar al operador de señales técnicas o de verificación, distinguiéndolas de un SOS	Servidor CAD (temporizador), Operador	RF23.1, RF23.2; HU42 (CA01–CA06), HU43 (CA01–CA06); DP19

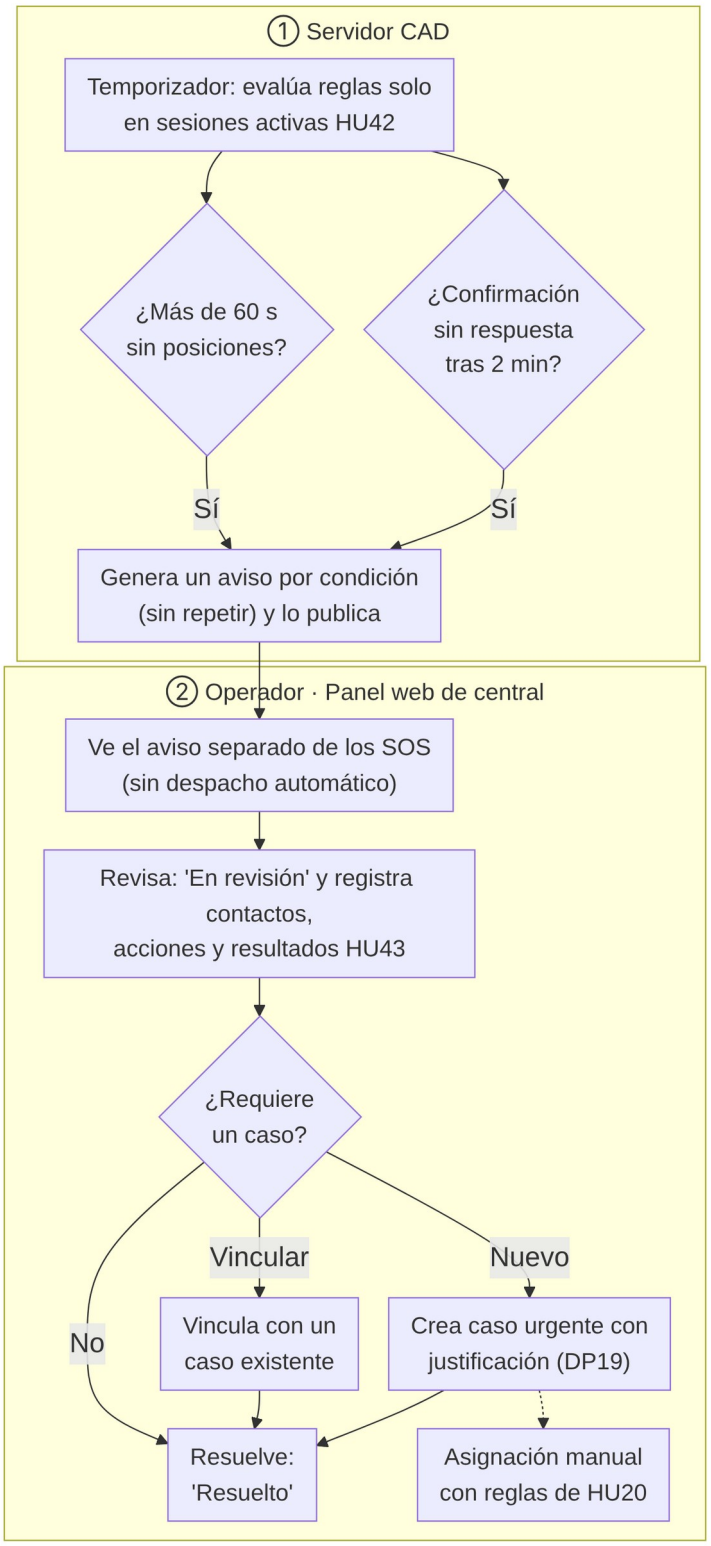


Figura 7. Avisos del acompañamiento (Mermaid)

Cómo leerlo. El temporizador evalúa las dos reglas documentadas (más de 60 s sin posiciones; confirmación sin respuesta durante 2 min) **solo en sesiones activas** (HU42-CA06). Genera un único aviso por condición mientras esta persista (HU42-CA03). El aviso aparece separado de los SOS y nunca dispara un despacho automático (HU42-CA04, HU43-CA06). Crear un caso urgente desde un aviso depende de **DP19** (origen y estado del nuevo caso), por lo que HU43-CA05 queda pendiente.

5.8. Diagramas de estados

Solo se dibujan transiciones presentes en los criterios de aceptación. Las transiciones ausentes (reasignación, rechazo del despacho, reapertura, efectos de DP03 y DP16) se indican como **no definidas**.

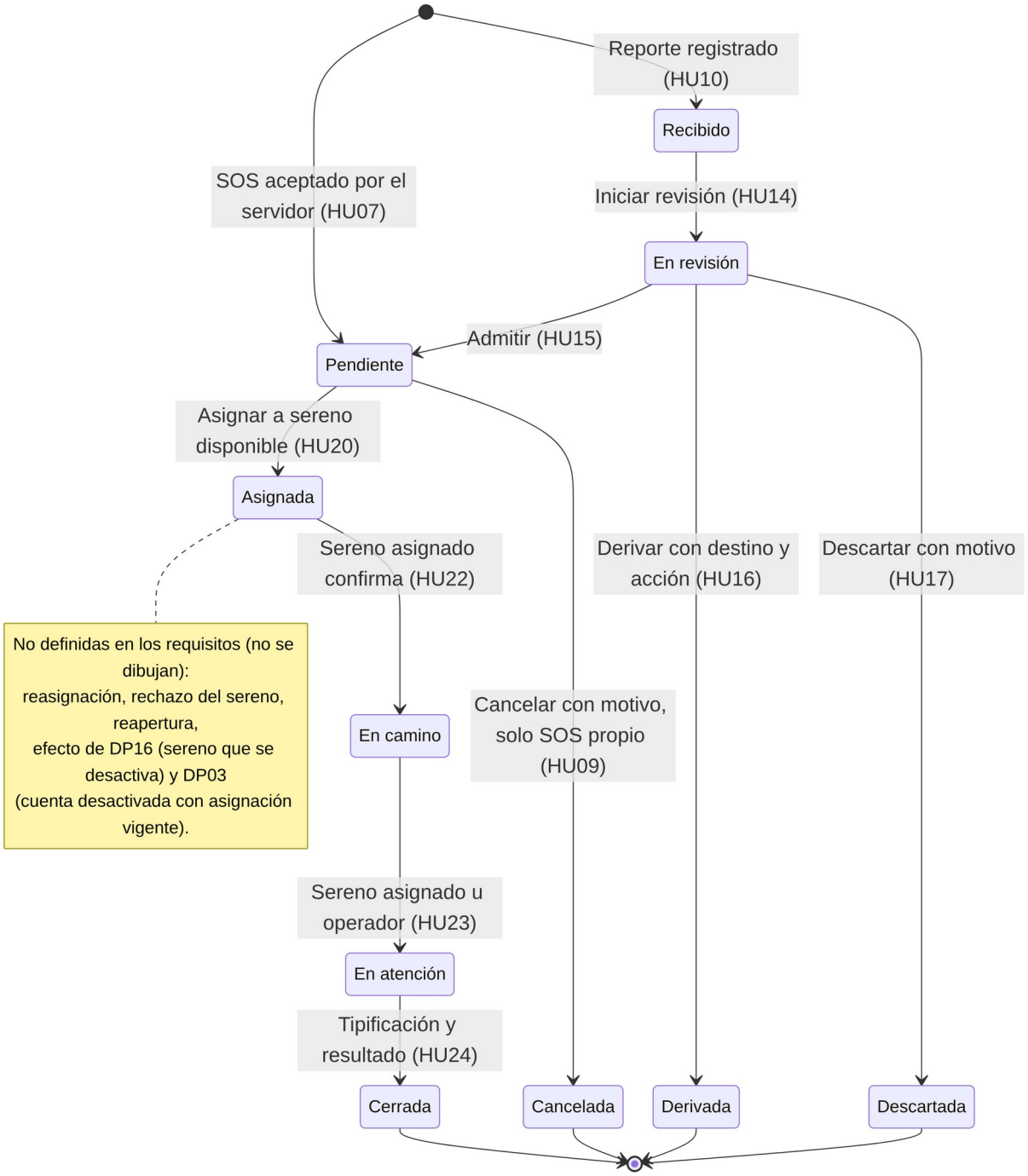


Figura 8. Ciclo de vida del caso (SOS y reporte) (Mermaid stateDiagram)

Cómo leerlo. Un SOS nace en "Pendiente"; un reporte, en "Recibido". Ambos convergen en "Pendiente" (el reporte, al ser admitido) y comparten el ciclo de despacho. "Cancelada" solo es alcanzable desde "Pendiente" y para SOS propios (HU09-CA01/CA03). "Derivada" y "Descartada" son finales y excluyen el caso de la asignación y del mapa de calor (HU17-CA04).

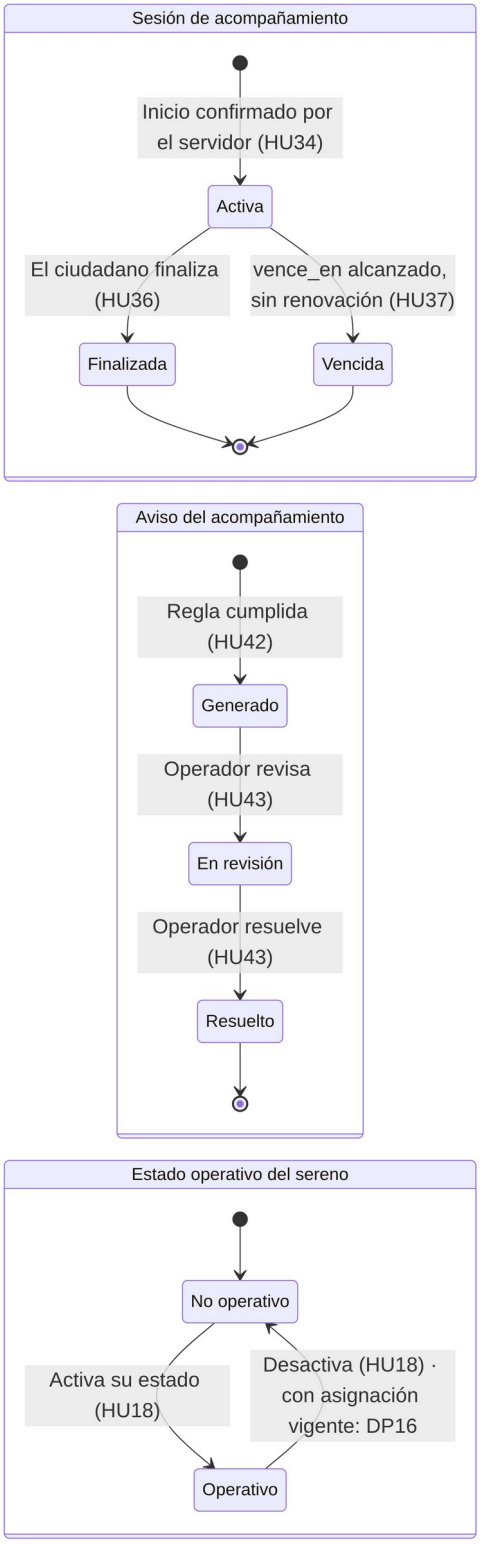


Figura 9. Estados de la sesión de acompañamiento, del aviso y del sereno (Mermaid stateDiagram)

Cómo leerlo. La sesión tiene dos finales distintos: "Finalizada" (decisión del ciudadano) y "Vencida" (servidor). Ninguno admite renovación (HU37-CA03). El aviso pasa por "Generado", "En revisión" y "Resuelto" (HU43). Para el sereno solo se modela el estado operativo. La **disponibilidad no es un estado almacenado**: se calcula como "operativo y sin asignación vigente" (HU19-CA02), lo que evita inconsistencias. La desactivación con asignación vigente depende de **DP16**.

6. Diagrama de capas

Se distinguen dos vistas: **dependencias del código** (quién conoce a quién al compilar, Figura 10) y **flujo de ejecución** (por dónde pasa una solicitud en tiempo de ejecución, Figura 11).

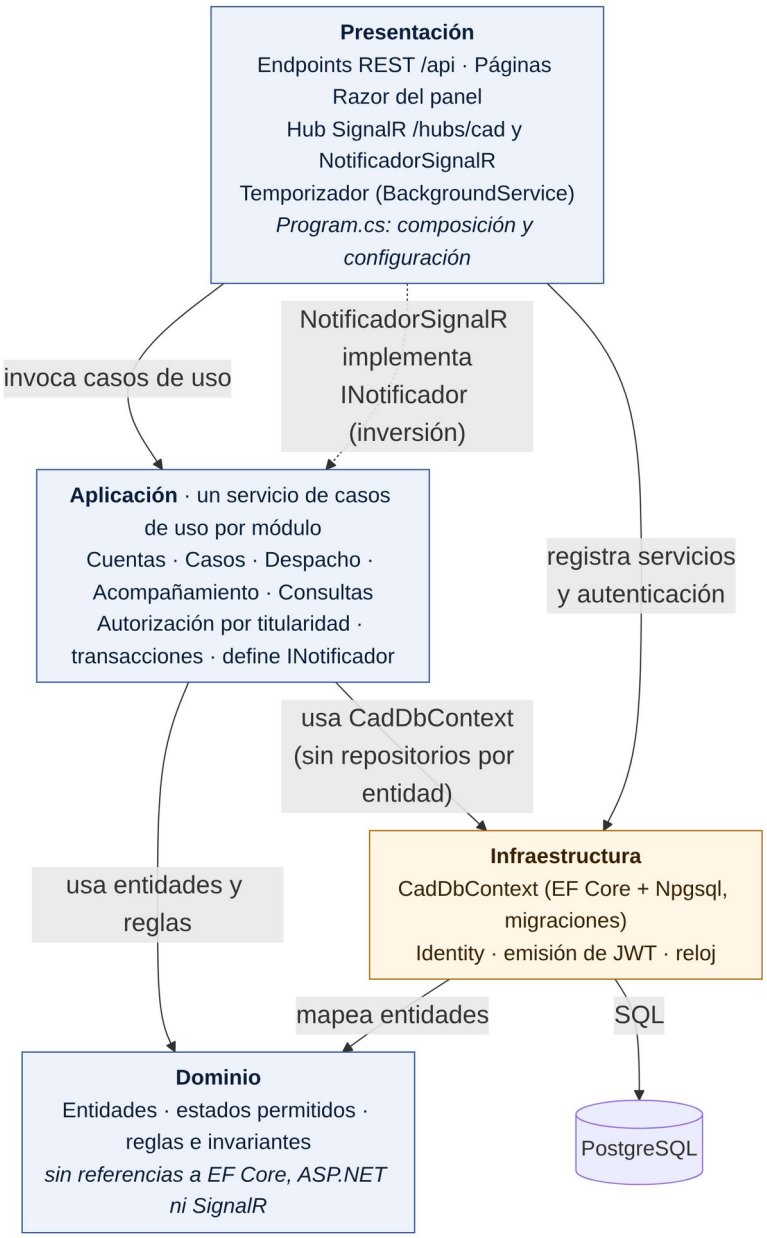


Figura 10. Capas del Servidor CAD y dependencias de código (Mermaid)

Cómo leerlo. Cada flecha continua es una dependencia de compilación permitida. El dominio no depende de nada externo. La aplicación usa directamente `CadDbContext` (EF Core ya es la unidad de trabajo), así que **no** hay repositorios por entidad. La única inversión de dependencia es `INotificador`, definida en la aplicación e implementada junto al Hub: así la aplicación no conoce SignalR y las pruebas pueden sustituir el notificador.

Tabla 20. Responsabilidad de cada capa

Capa	Contiene	No debe contener
Presentación	Endpoints, páginas Razor, Hub y NotificadorSignalR, temporizador, Program.cs, DTO de entrada y salida, traducción de errores a HTTP	Reglas de negocio, transacciones, consultas SQL de escritura

Capa	Contiene	No debe contener
Aplicación	Casos de uso por módulo (EmitirSos, Asignar, IniciarAcompanamiento…), autorización por titularidad, transacción, mapeo a DTO por rol, llamada a INotificador tras confirmar	Tipos HTTP, HttpContext, SignalR, HTML
Dominio	Entidades, enumeraciones de estado, métodos que validan transiciones e invariantes (por ejemplo, `Caso.Asignar()`), cálculo de vencimiento	Atributos o tipos de EF Core, acceso a base de datos, reloj del sistema (recibe la hora como parámetro)
Infraestructura	CadDbContext, configuración de entidades, migraciones, Identity, emisión de JWT, reloj	Decisiones de negocio

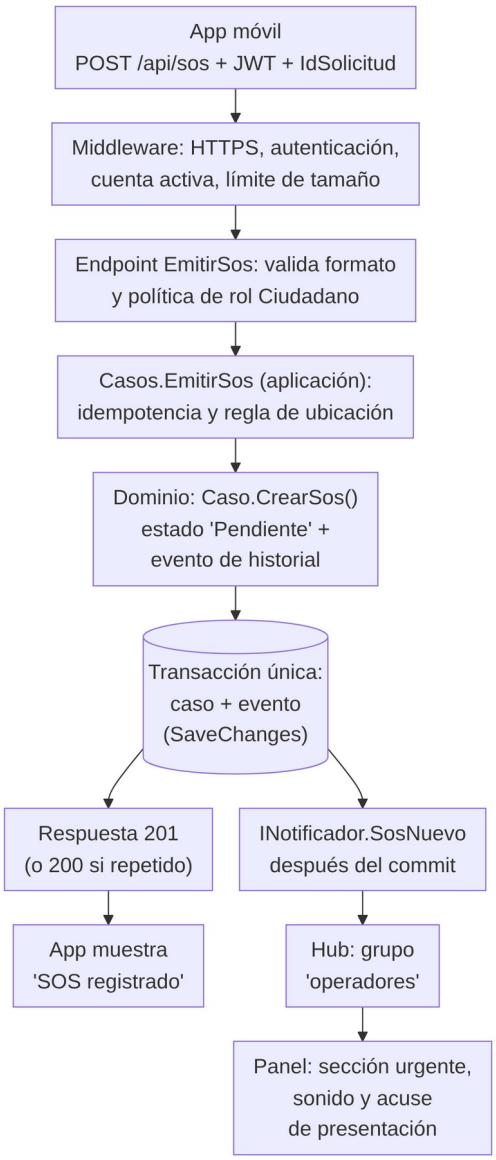


Figura 11. Flujo de ejecución de una operación representativa: emitir un SOS (Mermaid)

Cómo leerlo. La solicitud atraviesa el middleware (HTTPS, autenticación y comprobación de cuenta activa), el endpoint (formato y rol), el caso de uso (idempotencia y regla de ubicación) y el dominio (crea el caso "Pendiente" y su evento de historial). Después se confirma **una sola transacción**. A partir de ese punto hay dos ramas independientes: la respuesta HTTP a la app y la notificación al grupo "operadores". El panel muestra el SOS y envía un acuse, que sirve para medir RNF02.

7. Modelo C4

Diagramas en **PlantUML con C4-PlantUML**. Los nombres y límites son los mismos en todos los diagramas: *Sistema Integrado CAD*, *App móvil CAD*, *Panel web de central*, *Servidor CAD* y *Base de datos CAD*. Un **contenedor C4** es una unidad que ejecuta código o almacena datos; **no es un contenedor Docker**. Los módulos funcionales son componentes (C3), no contenedores. No se incluye C4 de código porque todavía no hay código.

Renderizador: PlantUML 1.2025.4 (Java 21) con la biblioteca estándar C4 incluida (`!include <C4/C4_Context>`, `<C4/C4_Container>`, `<C4/C4_Component>`, `<C4/C4_Deployment>`) y Graphviz (dot) para la disposición. Funciona sin conexión.

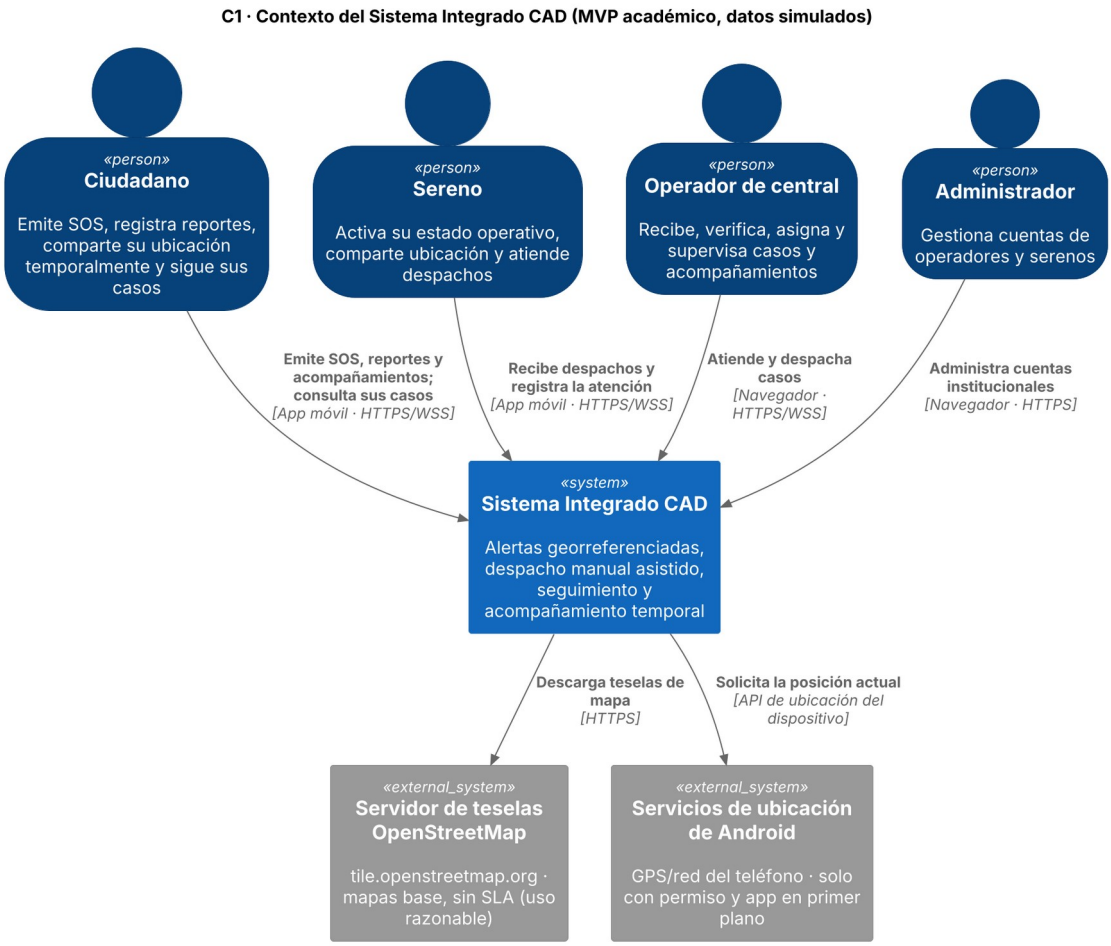


Figura 12. C1 · Contexto del sistema (C4-PlantUML)

Cómo leerlo. Las personas (azul oscuro) usan el sistema (azul) a través de la app o del navegador. Los sistemas externos (gris) son el servidor de teselas de OpenStreetMap y los servicios de ubicación de Android. No hay integraciones con entidades públicas (exclusión de [PS] 1.4). Cada relación indica su propósito y, entre corchetes, la tecnología o el protocolo.

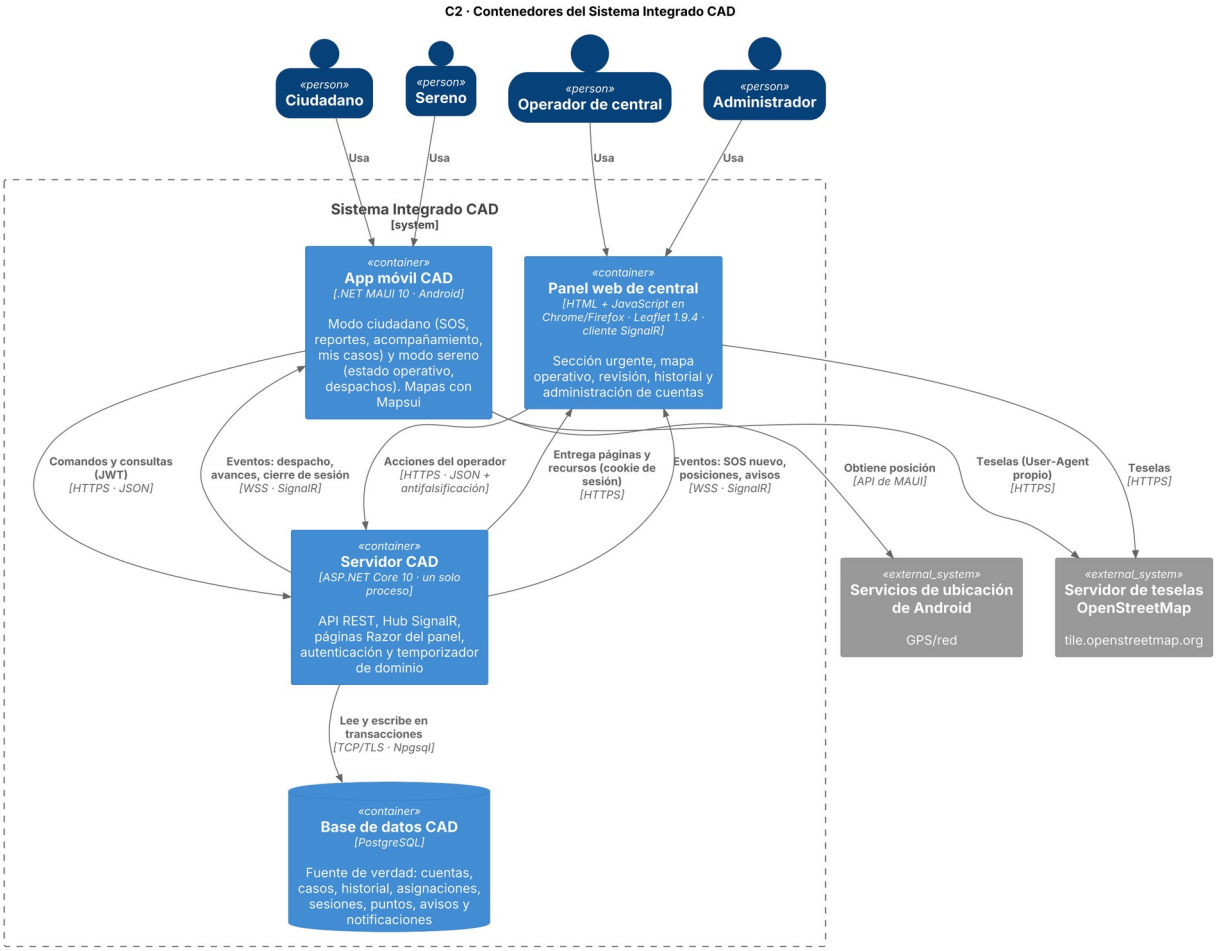


Figura 13. C2 · Contenedores (C4-PlantUML)

Cómo leerlo. Dentro del límite del sistema hay cuatro contenedores: dos clientes (app y panel), el Servidor CAD y la base de datos. El panel se dibuja aparte porque se ejecuta en el navegador, aunque lo entregue el Servidor CAD. Las relaciones distinguen las solicitudes de los clientes (HTTPS/JSON) de los eventos que envía el servidor (WSS/SignalR). Solo el Servidor CAD accede a la base de datos.

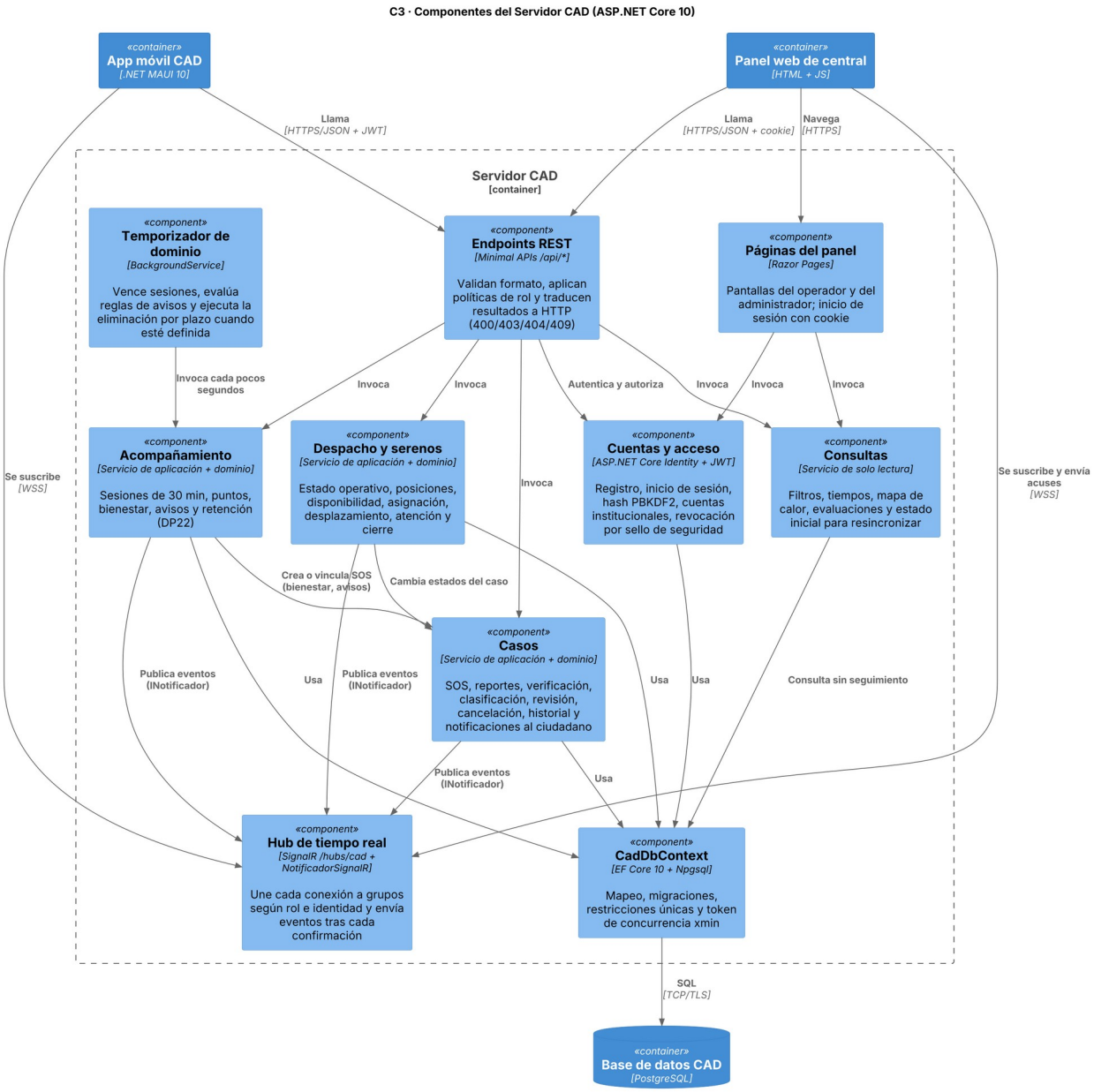


Figura 14. C3 · Componentes del Servidor CAD (C4-PlantUML)

Cómo leerlo. Los componentes de entrada (endpoints, páginas, Hub y temporizador) delegan en los módulos de aplicación (Cuentas y acceso, Casos, Despacho y serenos, Acompañamiento y Consultas). Los módulos publican eventos a través del Hub y usan CadDbContext. Despacho modifica casos solo mediante el módulo Casos, y Acompañamiento crea o vincula SOS mediante Casos. Consultas solo lee.

8. Interacciones y despliegue

8.1. Secuencia del flujo técnico más importante: SOS hasta la central

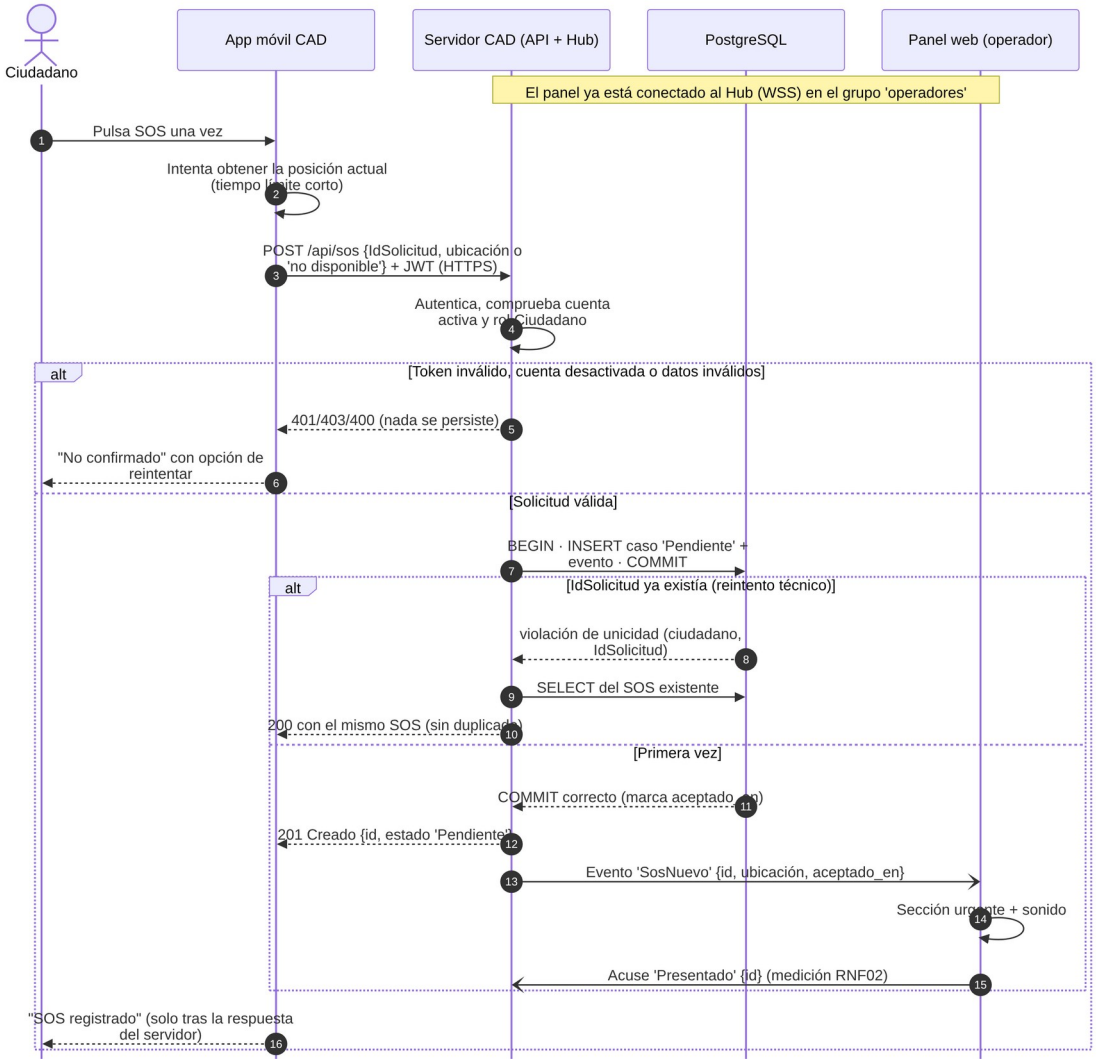


Figura 15. Secuencia: emisión del SOS, persistencia, confirmación y notificación en tiempo real (Mermaid)

Cómo leerlo. La secuencia separa tres hechos que no deben confundirse: (a) **persistido**: el COMMIT en PostgreSQL (paso 11); (b) **confirmado al cliente**: la respuesta 201/200 (pasos 10 y 12), tras la cual la app muestra "SOS registrado"; (c) **notificado**: el evento SignalR al panel (paso 13), que se emite solo después del COMMIT. Recibir un mensaje por WebSocket **no** equivale a haber persistido una operación. El acuse del panel (paso 15) sirve para medir RNF02 desde `aceptado_en` hasta la recepción del acuse, una cota superior del tiempo real de presentación.

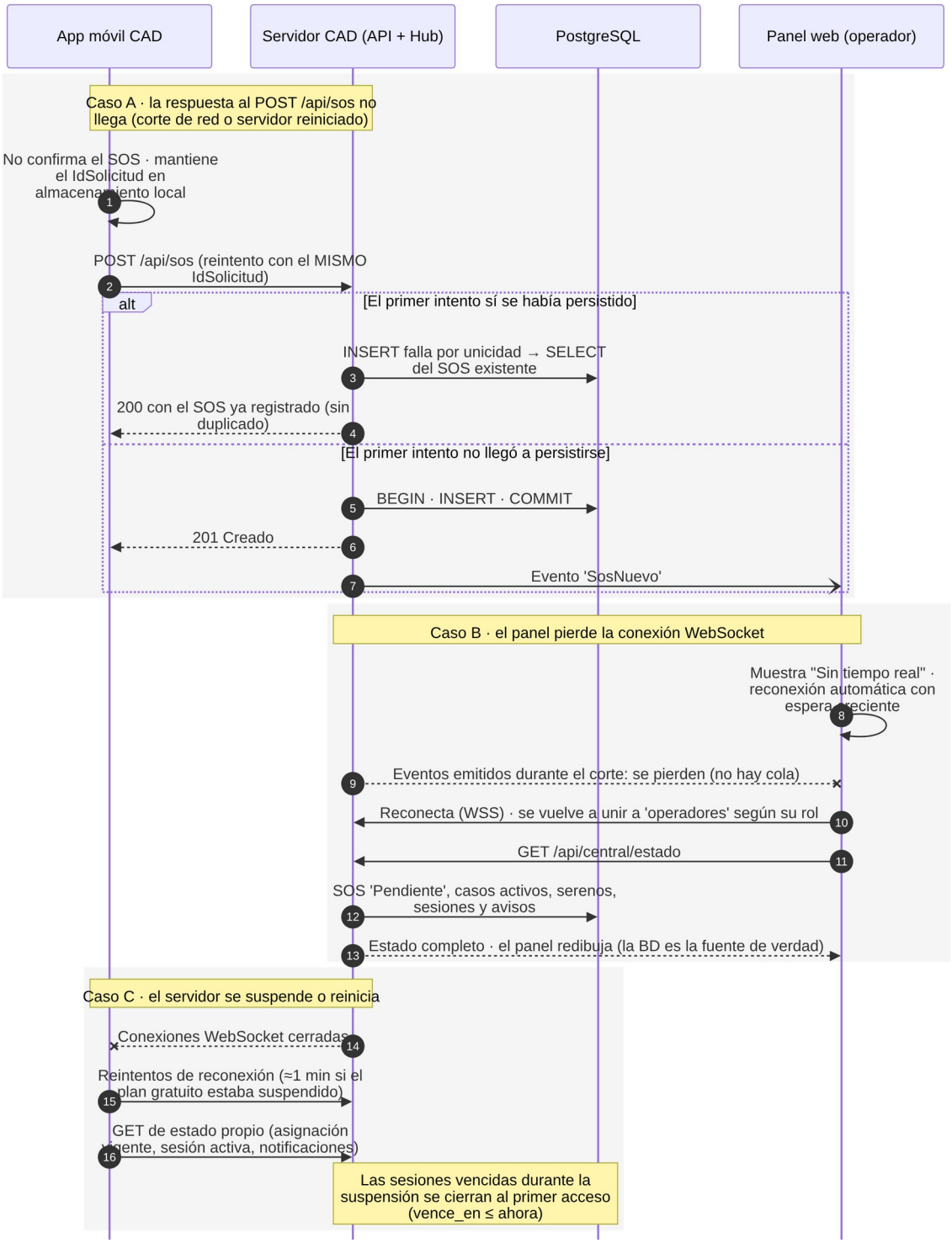


Figura 16. Secuencia: fallo de conexión y recuperación (Mermaid)

Cómo leerlo. Caso A: si la respuesta no llega, la app reintenta con el mismo IdSolicitud y el servidor nunca duplica. Caso B: los eventos emitidos durante un corte **se pierden** (SignalR no tiene cola persistente); al reconectar, el panel consulta `/api/central/estado` y redibuja a partir de la base de datos. Caso C: si el servidor se suspende o reinicia, los clientes reconectan y consultan su estado; las sesiones que vencieron mientras tanto se cierran en el primer acceso porque `vence_en` está persistido.

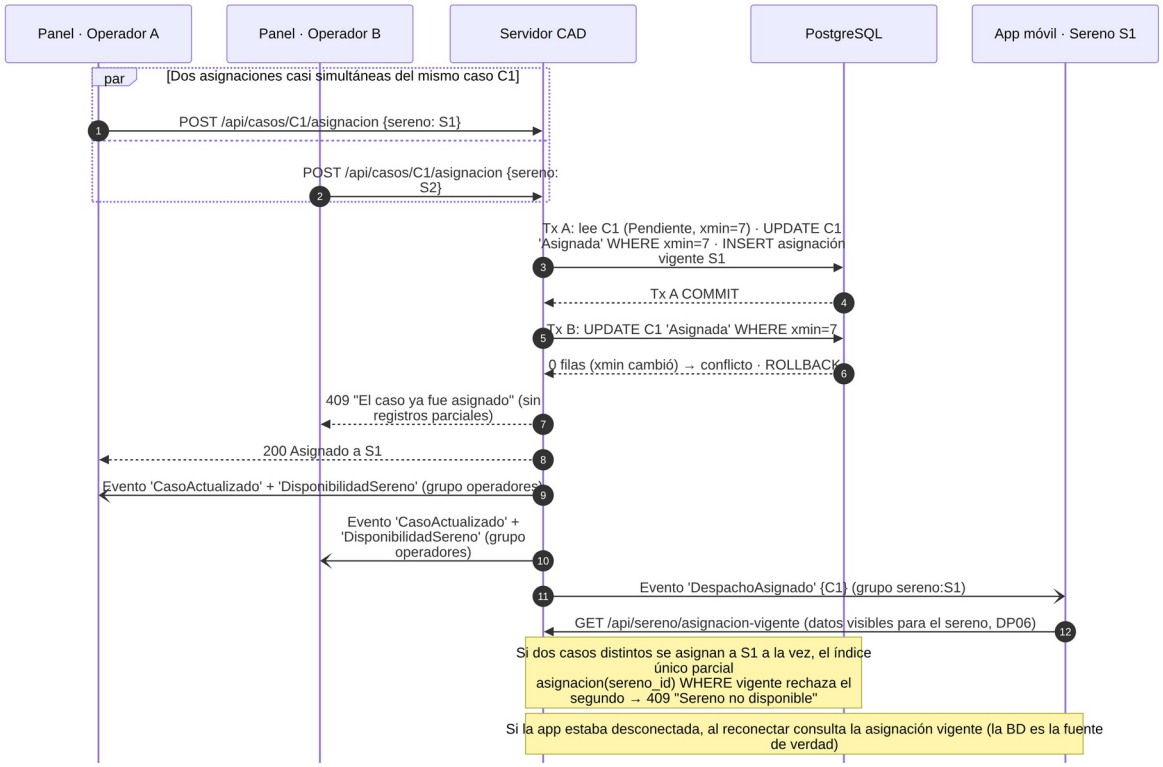


Figura 17. Secuencia: asignaciones concurrentes del mismo caso (Mermaid)

Cómo leerlo. Las dos transacciones leen el caso con el mismo token `xmin`. La primera confirma; la segunda no actualiza ninguna fila porque el token cambió, hace ROLLBACK y responde 409 sin dejar registros parciales (HU20-CA04, CA08). Si se asignan dos casos distintos al mismo sereno a la vez, lo impide el índice único parcial de asignaciones vigentes (HU20-CA05).

8.2. Despliegue mínimo

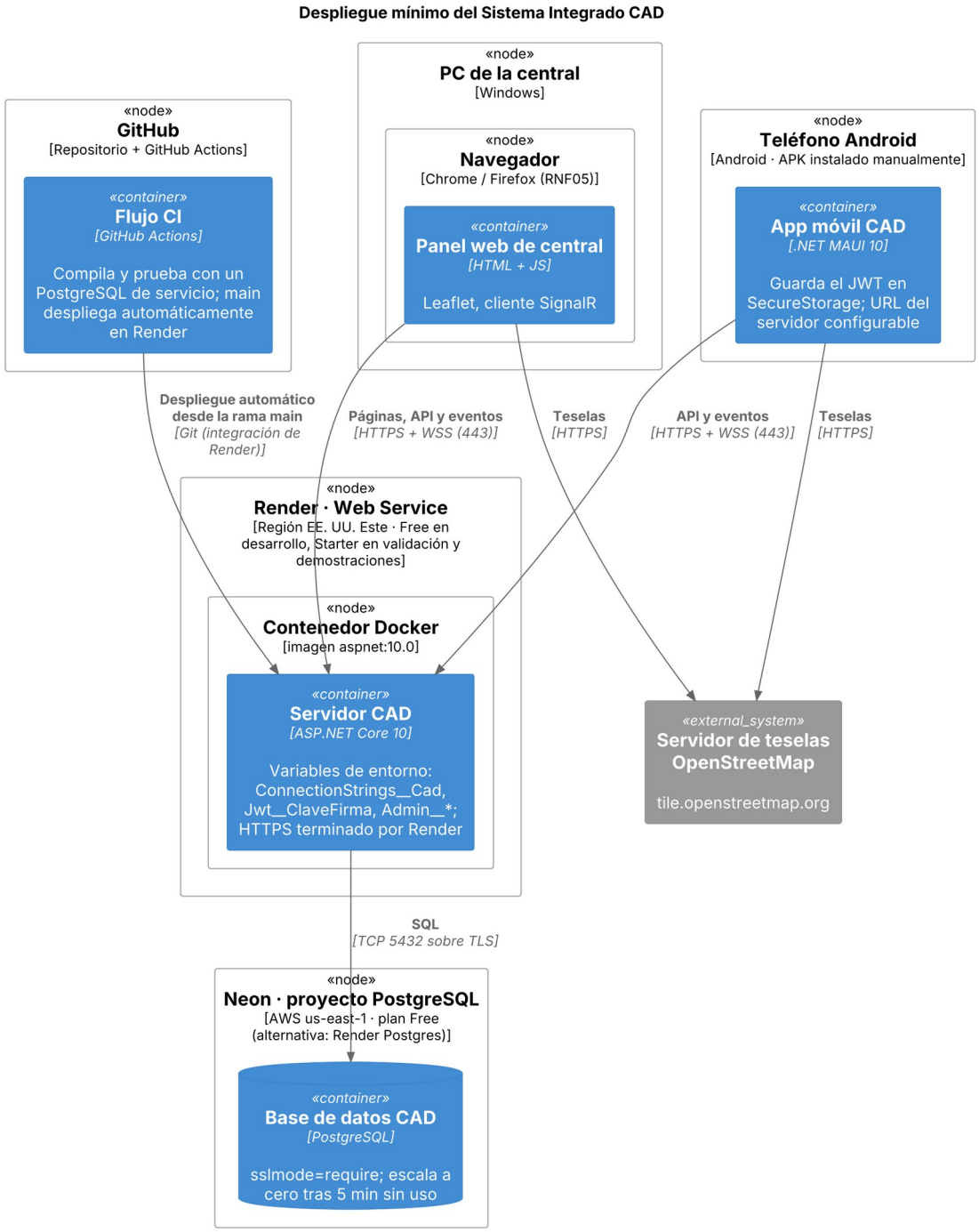


Figura 18. Despliegue mínimo propuesto (C4-PlantUML, Deployment)

Cómo leerlo. Hay tres entornos de ejecución: el **teléfono Android** (app), el **navegador** del PC de la central (panel) y **Render** (Servidor CAD en un contenedor Docker), además de **Neon** (PostgreSQL). Todo el tráfico de clientes entra por el puerto 443 con HTTPS y WSS. Render termina TLS y el Servidor CAD se conecta a Neon con TLS (`sslmode=require`). GitHub despliega automáticamente la rama `main` en Render.

Tabla 21. Configuración de secretos

Secreto	Dónde se guarda	Nunca en
Cadena de conexión de PostgreSQL	Variable de entorno `ConnectionStrings__Cad` en Render; `.env` local fuera del control de versiones	Repositorio, appsettings.json, logs
Clave de firma JWT (≥256 bits)	Variable de entorno `Jwt__ClaveFirma`	Código de la app móvil

Secreto	Dónde se guarda	Nunca en
Credenciales del administrador inicial (DP02)	`Admin__Email` y `Admin__ClaveInicial`; cambio obligatorio tras el primer acceso (supuesto)	Repositorio
Claves de Data Protection (cookies)	Tabla en PostgreSQL (`PersistKeysToDbContext`), para que las cookies sobrevivan a los redespliegues	Sistema de archivos del contenedor (efímero)
Token en la app	SecureStorage de MAUI (Android Keystore)	Preferencias en texto plano

8.3. Efecto de la suspensión del alojamiento

Tabla 22. Impacto de la suspensión de Render Free

Aspecto	Efecto si el servidor está suspendido	Mitigación
Conexiones persistentes	Se cortan; tras ~1 min de arranque los clientes reconectan	Reconexión automática y resincronización (Figura 16)
RNF02 (≤3 s)	El primer SOS tras la suspensión tarda lo que dura el arranque: incumple	Starter durante las pruebas y demos; calentar el servicio antes
RNF10 (99 % en 8 h)	Cada suspensión y arranque cuenta como interrupción; además, Free puede reiniciarse sin aviso	Prueba de 8 h solo con Starter
Vencimiento de sesiones (HU37)	El temporizador no se ejecuta, pero el estado no se corrompe: el vencimiento se evalúa con `vence_en` en el primer acceso	Evaluación diferida + temporizador
Avisos (HU42)	No se generan mientras está suspendido	Durante una sesión activa llegan posiciones cada 15 s, lo que mantiene el servidor despierto; en validación se usa Starter
Eliminación por plazo (RNF13)	Se retrasa hasta el siguiente arranque	La tarea se ejecuta al arrancar y luego periódicamente
Base de datos Neon	Escala a cero tras 5 min; la primera consulta tiene latencia adicional	Medir en SP-D; alternativa Render Postgres

9. Datos, seguridad y fiabilidad

9.1. Entidades principales y relaciones

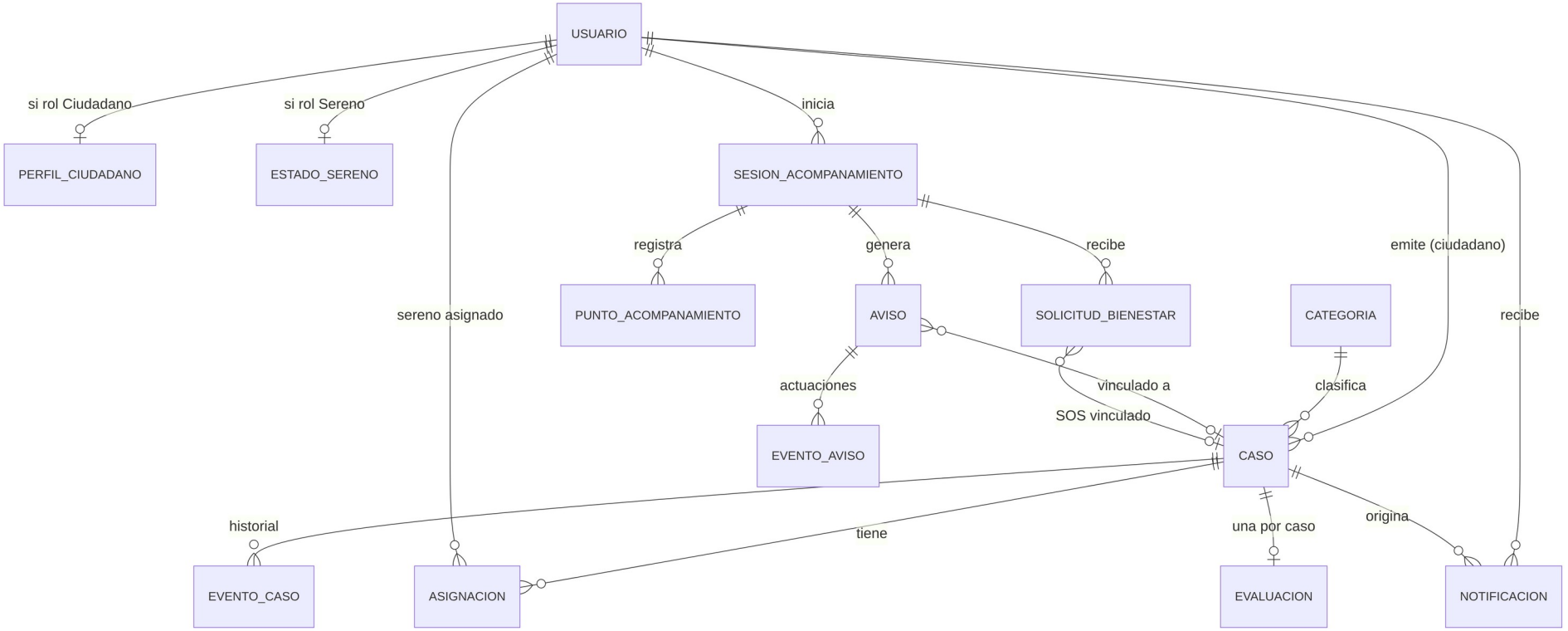


Figura 19. Modelo de datos conceptual (Mermaid erDiagram)

Cómo leerlo. Notación "pata de gallo": `||` = exactamente uno; `o|` = cero o uno; `o{` = cero o muchos. USUARIO agrupa a los cuatro roles (ASP.NET Core Identity); el perfil del ciudadano y el estado del sereno son extensiones uno a uno. CASO es común a SOS y reportes (columna `origen`). El historial de actuaciones (EVENTO_CASO) está separado de los recorridos (PUNTO_ACOMPANAMIENTO), como exigen HU27-CA05 y HU39-CA01. Del sereno solo se guarda la **última** posición en ESTADO_SERENO: ningún requisito pide su historial y así se reduce la exposición de datos.

Tabla 23. Atributos y restricciones relevantes

Entidad	Atributos clave	Restricciones de integridad
Caso	origen, estado, ciudadano_id, id_solicitud, lat/lon/precisión/obtenida_en (nulos si no disponible), referencia_texto, categoría original y actual, resultado_público, notas_internas, creado_en	UNIQUE (ciudadano_id, id_solicitud); CHECK de lat −90…90 y lon −180…180; CHECK de que coordenadas y precisión son todas nulas o todas presentes; token xmin
Asignacion	caso_id, sereno_id, operador_id, vigente, asignada_en, finalizada_en	UNIQUE (caso_id) WHERE vigente; UNIQUE (sereno_id) WHERE vigente
EventoCaso	tipo, actor_id, visibilidad (pública o interna), datos (jsonb), ocurrido_en	Solo inserción (no se actualiza ni se borra); base de HU27 y HU28
SesionAcompanamiento	ciudadano_id, id_solicitud, estado, inicio, vence_en, cerrada_en, motivo_cierre	UNIQUE (ciudadano_id) WHERE estado = 'Activa'; UNIQUE (ciudadano_id, id_solicitud); CHECK vence_en = inicio + 30 min
PuntoAcompanamiento	sesion_id, lat, lon, precisión, obtenida_en (dispositivo), recibida_en (servidor), conservación excepcional	UNIQUE (sesion_id, obtenida_en) para descartar reenvíos; se inserta solo si la sesión está activa y no vencida
SolicitudBienestar	sesion_id, operador_id, solicitada_en, presentada_en, respuesta, respondida_en, caso_vinculado_id	Respuesta ∈ {Estoy bien, Necesito ayuda}; una respuesta por solicitud
Aviso / EventoAviso	sesion_id, regla, estado, generado_en, caso_vinculado_id; actuaciones	UNIQUE (sesion_id, regla) WHERE condición abierta (HU42-CA03)
Notificacion	usuario_id, caso_id, tipo, vista_previa (sin datos sensibles), leída, creada_en	Solo para el titular del caso (HU31-CA04)
Evaluacion	caso_id, puntuación, comentario	UNIQUE (caso_id); CHECK puntuación 1…5
Usuario / PerfilCiudadano / EstadoSereno	Identity: identificador único, hash, rol, activo, sello de seguridad; teléfono (DP01); operativo y última posición	Identificador único normalizado (HU01-CA04/CA05)
MedicionPresentacion (solo pruebas)	caso_id, aceptado_en, acuse_en	Se activa por configuración durante la prueba de RNF02

9.2. Fuente de verdad

Decisión propuesta: PostgreSQL es la única fuente de verdad. SignalR y la memoria del proceso son solo mecanismos de entrega. Si un estado mostrado en el panel o en la app contradice la base de datos, prevalece la base de datos, y los clientes se resincronizan al reconectar. La disponibilidad del sereno se **calcula** (operativo y sin asignación vigente) y no se almacena como indicador.

9.3. Operaciones que necesitan transacción

Tabla 24. Límites transaccionales por operación

Operación	Contenido de la transacción	Protección frente a concurrencia o repetición
Registrar ciudadano (HU01)	Usuario + perfil	Índice único de identificador (CA05)

Operación	Contenido de la transacción	Protección frente a concurrencia o repetición
Emitir SOS (HU07/HU08)	Caso "Pendiente" + evento	UNIQUE (ciudadano, IdSolicitud) → devuelve el existente
Registrar reporte (HU10)	Caso "Recibido" + categoría original + evento	Ídem
Transiciones de revisión (HU14–HU17)	Caso (nuevo estado y responsable) + evento + notificación pública si corresponde (DP05)	Token xmin: la segunda decisión simultánea recibe 409
Asignar (HU20)	Caso "Asignada" + asignación vigente + evento + notificación	xmin + índices únicos parciales (caso y sereno)
Desplazamiento, inicio y cierre (HU22–HU24)	Caso + evento (+ asignación no vigente al cerrar) + notificación	xmin; repetir la misma transición con el mismo actor devuelve el estado actual sin crear otro evento (HU22-CA05)
Cancelar SOS (HU09)	Caso "Cancelada" + motivo + evento	xmin frente a una asignación simultánea (CA06); una repetición no altera motivo ni hora (CA08)
Iniciar acompañamiento (HU34)	Cerrar sesiones vencidas del ciudadano + insertar la nueva sesión	UNIQUE parcial de sesión activa + IdSolicitud
Registrar punto (HU35)	Comprobar sesión activa y no vencida + insertar el punto	UNIQUE (sesión, obtenida_en)
Finalizar o vencer (HU36/HU37)	Sesión "Finalizada" o "Vencida" + hora de cierre	Actualización condicionada a estado = Activa; idempotente
"Necesito ayuda" (HU41)	Respuesta + nuevo SOS o vínculo con el SOS vigente	IdSolicitud de la respuesta; regla de "SOS vigente" pendiente (D9)
Generar aviso (HU42)	Aviso + evento	UNIQUE parcial (sesión, regla) mientras la condición esté abierta
Evaluar (HU32)	Evaluación	UNIQUE (caso_id)
Desactivar cuenta (HU05)	Usuario inactivo + nuevo sello de seguridad	El efecto sobre sesiones y asignaciones depende de DP03

Toda notificación por SignalR se envía **después** del COMMIT. Si el envío falla, el dato ya está guardado y los clientes lo recuperan al resincronizarse. No hace falta un *outbox* porque la base de datos ya es la fuente que consultan los clientes.

9.4. Idempotencia y concurrencia

- **Creación** (SOS, reporte, sesión, respuesta de bienestar): el cliente genera un `IdSolicitud` (UUID) por acción del usuario y lo reutiliza en cada reintento. La restricción única lo convierte en idempotente.
- **Transiciones de estado:** control optimista con `xmin`. El dominio valida el estado de origen; si otra transacción modificó el caso, se devuelve 409 con el estado actual.
- **Recursos únicos por diseño:** índices únicos parciales para la asignación vigente (por caso y por sereno), la sesión activa por ciudadano y el aviso abierto por regla.
- **Prueba:** las pruebas de integración lanzan solicitudes paralelas (`Task.WhenAll`) contra PostgreSQL real y comprueban que no quedan registros parciales.

9.5. Autorización por rol y por titularidad

Tabla 25. Matriz resumida de autorización

Operación	Ciudadano	Operador	Sereno	Admin.
Emitir SOS, reportar, acompañamiento, responder bienestar	Sí (propios)	No	No	No

Operación	Ciudadano	Operador	Sereno	Admin.
Consultar mis casos y notificaciones; cancelar SOS; evaluar	Solo titular	No	No	No
Sección urgente, verificación, clasificación, revisión, asignación, historial, filtros, tiempos	No	Sí	No	No
Consultar acompañamientos, posiciones e historial de sesiones	No	Sí ("autorizado", DP04)	No	No
Estado operativo y posición del sereno	No	Consulta	Sí (propio)	No
Consultar despacho; confirmar desplazamiento	No	No (HU22-CA04)	Solo asignado	No
Inicio de atención y cierre	No	Sí	Solo asignado	No
Gestionar cuentas institucionales	No	No	No	Sí

La autorización se comprueba **en el servidor** para cada endpoint y para cada grupo del Hub. Un cliente no puede unirse a grupos arbitrarios: el servidor lo asigna al conectarse según los *claims* del token o de la cookie. Una matriz de pruebas automatizada (endpoint × rol) verifica HU03-CA04 de forma acumulativa.

9.6. Credenciales, sesiones y cifrado

- **Contraseñas:** PasswordHasher de Identity (formato V3: PBKDF2-HMAC-SHA512, sal de 128 bits, 100 000 iteraciones por defecto), documentado para RNF01.
- **Panel:** cookie HttpOnly, Secure y SameSite; protección antifalsificación en formularios y en las llamadas fetch.
- **App:** JWT firmado (HS256) con vida configurable y sin token de refresco. En cada solicitud se comprueba que la cuenta siga activa y que el sello de seguridad no haya cambiado (20 usuarios: costo despreciable). Esto permite revocar sesiones cuando se resuelva DP03.
- **Cifrado:** HTTPS y WSS terminados por Render, con HSTS; Android bloquea por defecto el tráfico sin cifrar (solo se permite en la configuración de depuración local). Conexión con PostgreSQL mediante TLS (RNF09).
- **Abuso:** limitador de solicitudes integrado de ASP.NET Core en el inicio de sesión (opcional, de bajo costo).
- **CORS:** no se necesita (el panel tiene el mismo origen y la app no está sujeta a CORS); se deja deshabilitado.

9.7. Información que no debe aparecer en registros

- Contraseñas, hashes, tokens JWT, cookies y el parámetro `access_token` que SignalR envía en la URL de WebSocket: se registra la ruta **sin** la cadena de consulta.
- Coordenadas de ciudadanos, recorridos y precisión (RNF07, RNF13).
- Teléfonos, descripciones de reportes, notas internas, motivos y comentarios.
- Cuerpos de solicitudes y respuestas; `EnableSensitiveDataLogging` de EF Core desactivado.
- Se permite: método, ruta, estado HTTP, duración, identificador interno (UUID) del usuario y del caso, y tipo de evento.

9.8. Reconexión y recuperación del estado

- **Panel:** reconexión automática de SignalR; indicador visible "Sin tiempo real"; al reconectar, GET `/api/central/estado` (SOS pendientes, casos activos, serenos, sesiones y avisos).
- **App del sereno:** al abrir o reconectar, consulta la asignación vigente.

- **App del ciudadano:** al abrir o reconectar, consulta la sesión activa, sus casos y las notificaciones no leídas (HU31-CA02). Las operaciones pendientes (cierre de acompañamiento, SOS sin confirmar) se guardan localmente con su IdSolicitud y se reintentan.
- **Servidor:** al arrancar recalcula en memoria si hay sesiones o avisos activos (para el temporizador). Los grupos de SignalR se reconstruyen en cada conexión.

9.9. Tareas de vencimiento y eliminación

Tabla 26. Tareas programadas dentro del proceso

Tarea	Frecuencia	Regla	Estado
Vencer sesiones de acompañamiento	Cada 5 s (parámetro) mientras existan sesiones activas, más una comprobación al acceder	vence_en ≤ ahora → "Vencida" + evento a la app (HU37)	Definida
Aviso por posición desactualizada	Cada 5 s si hay sesiones activas	Más de 60 s sin puntos (HU42-CA01)	Definida
Aviso por confirmación sin respuesta	Cada 5 s si hay solicitudes presentadas	2 min sin respuesta (HU42-CA02)	Definida
Eliminar puntos vencidos	Al arrancar y cada hora (parámetro)	Plazo de DP22; tratamiento separado de los puntos con conservación excepcional	Bloqueada (DP22): parámetro vacío y tarea deshabilitada

Pendiente: No se propone ningún plazo de conservación de recorridos. HU39-B y RNF13 no pueden aceptarse mientras DP22 no esté resuelta ([PS] S4-T14).

10. Justificación y trazabilidad

Tabla 27. Trazabilidad: requisito → decisión → validación

Requisito o restricción	Decisión arquitectónica	Beneficio	Costo o desventaja	Forma de validación
RNF03 (20 sesiones)	Un solo Servidor CAD, una instancia	Sin coordinación distribuida	Sin tolerancia a fallos del proceso	Prueba de 30 min con Cad.PruebaCarga
RNF02 (≥95 % en ≤3 s)	SignalR tras el COMMIT + acuse del panel	Entrega en milisegundos en la misma región	Requiere conexión abierta y servidor despierto	≥100 SOS con 20 sesiones; informe de aceptado→acuse
RNF08, HU20-CA04/05	Token xmin + índices únicos parciales	Garantía en la base de datos, no en el código	La interfaz debe manejar 409	Pruebas paralelas de integración
RNF08, HU07-CA07, HU10-CA08, HU34-CA06, HU41-CA07	IdSolicitud único por acción del cliente	Reintentos seguros	Una columna más y lógica de reintento	Reenvío del mismo cuerpo: un solo registro
RNF06, HU07-CA03	Confirmación solo tras la respuesta del servidor	Honestidad de la interfaz	Espera visible con red lenta	Prueba con red cortada antes de la respuesta
RNF12, HU03-CA03	Autorización en el servidor (políticas + titularidad) y DTO por rol	Resiste solicitudes directas	Más DTO	Matriz automatizada endpoint × rol
RNF07, RNF11, [PS] 1.5	Captura solo con la app en primer plano; transmisión solo con sesión activa en el servidor	No promete lo que no se valida	Sin seguimiento con la app cerrada	Pruebas manuales + conteo de llamadas a endpoints de posición
HU37-CA01	vence_en persistido + temporizador + comprobación al acceder	Correcto aunque el servidor duerma	El evento de cierre puede llegar con retraso si estuvo suspendido	Sesión con el dispositivo desconectado hasta los 30 min
HU42	Temporizador en el proceso	Sin cron ni workers (no existen en Render Free)	Solo funciona con el servidor despierto	Sesión sin puntos > 60 s; solicitud sin respuesta > 2 min
RNF13, DP22	Plazo como parámetro; tarea deshabilitada	No inventa reglas	HU39-B bloqueada	Con un plazo de prueba configurado
RNF09	HTTPS/WSS de Render, HSTS, Android sin texto plano, TLS a PostgreSQL	Sin configuración de certificados propia	Depende del proveedor	Inspección de URL y pruebas de conexión sin cifrar
RNF01	PasswordHasher de Identity (PBKDF2-HMAC-SHA512)	Implementación estándar y documentada	Menos resistente a GPU que Argon2	Inspección de base de datos y logs
RNF10	Render Starter en la ventana de validación	Sin suspensión	USD 7/mes prorrateado	Monitor de 8 h (acceso, registro, panel)
Presupuesto mínimo [EQ]	Render Free + Neon Free en desarrollo	USD 0	Suspensión 15 min; cuota de 100 CU-h	Seguimiento del consumo en Neon
App nativa [EQ]	.NET MAUI 10 (Android)	Un lenguaje y contratos compartidos	Herramientas Android y compilación lentas	Spikes SP-A, SP-B, SP-C y SP-F
C#/.NET [EQ]	ASP.NET Core 10 LTS	Soporte hasta 2028	Ninguna relevante	—
RF05.2, HU19, HU25; [PS] 1.4	Leaflet + Mapsui + teselas OSM	Libre, sin clave	Política de uso, sin SLA	Atribución visible; User-Agent
RF18.2, HU31-CA02	Notificaciones persistidas + envío si hay conexión	Nada se pierde al estar desconectado	Tabla adicional	Ciudadano desconectado que vuelve a entrar

Requisito o restricción	Decisión arquitectónica	Beneficio	Costo o desventaja	Forma de validación
RF12.1, RF13, HU27, HU28	EventoCaso único de solo inserción	Historial y tiempos desde la misma fuente	Consultas de agregación	Pruebas de historial y tiempos
RNF14	GitHub con ramas, PR y CI	Revisión y pruebas automáticas	Disciplina del equipo	Revisión del repositorio

10.1. Por qué esta arquitectura es suficiente

Los requisitos más exigentes son RNF02, RNF03 y RNF08. RNF02 depende de la latencia de un mensaje dentro de una misma región, no del volumen. RNF03 fija 20 sesiones, una carga que un proceso de ASP.NET Core maneja con amplio margen. RNF08 depende de la atomicidad y de las restricciones de una base de datos relacional. Las tres se cumplen con un proceso y una base de datos. El resto de requisitos son funcionales y de seguridad, y se resuelven con el framework sin componentes adicionales.

10.2. Por qué una solución más compleja no compensa

- **Microservicios o servicios separados:** convertirían RNF08 (hoy una transacción local) en un problema distribuido y multiplicarían el costo de alojamiento para evitar las suspensiones.
- **Redis o un backplane de SignalR:** solo son necesarios con varias instancias del servidor, y no hay ninguna.
- **Broker de mensajes u outbox:** los clientes se resincronizan desde la base de datos; un evento perdido no pierde información.
- **PostGIS:** no hay búsquedas por proximidad (el operador compara en el mapa, HU20-CA01) y el mapa de calor se calcula en el cliente.
- **SPA separada:** añade otro proyecto, otro lenguaje y la gestión de tokens en el navegador sin que lo pida ningún requisito.

10.3. Limitaciones aceptadas

- Un único punto de fallo (el proceso servidor); la disponibilidad solo se valida durante 8 h controladas.
- Sin notificaciones con la app cerrada y sin seguimiento en segundo plano.
- Eventos de tiempo real no garantizados; se garantiza la resincronización.
- Solo Android (no hay iOS).
- Sin copias de seguridad gestionadas en las capas gratuitas (datos simulados).

10.4. Fuera del MVP

Ruteo y cálculo de cercanía, despacho automático, integraciones con entidades públicas, telefonía, notificaciones push, seguimiento en segundo plano, iOS, alta disponibilidad, copias de seguridad con retención y auditoría formal de seguridad.

10.5. Cambios futuros que justificarían revisar la arquitectura (hipótesis)

Tabla 28. Hipótesis de evolución

Hipótesis (no es una necesidad actual)	Revisión que justificaría
Más de una instancia del servidor por carga o disponibilidad	Backplane de SignalR (por ejemplo, Redis) y temporizador con exclusión (bloqueo asesor de PostgreSQL)
Recepción del despacho con la app cerrada	Notificaciones push (FCM) y servicio en primer plano de Android para el sereno
Búsqueda automática del sereno más cercano	PostGIS e índices espaciales
Uso real con datos personales	Copias de seguridad, cifrado en reposo administrado, auditoría, política de retención aprobada y marco legal
Integración con entidades externas	Módulo de integración con cola de salida y reintentos

10.6. Registros de decisiones arquitectónicas (ADR)

ADR-01. Aplicación móvil nativa con .NET MAUI en lugar de PWA

Elemento	Contenido
Contexto	[PS] 1.4 preveía PWA para ciudadano y sereno. El 04/10/2026 [EQ] pidió una aplicación, preferentemente con framework nativo, para Android; el backend será C#/.NET.
Decisión	Una sola app .NET MAUI 10 para Android con dos modos (ciudadano y sereno) según el rol. Comparte con el servidor el proyecto Cad.Contratos. Mapas con Mapsui y teselas OSM; tiempo real con el cliente .NET de SignalR.
Alternativas descartadas	PWA (no cumple la preferencia de [EQ]); Kotlin + Jetpack Compose (otro lenguaje, sin código compartido); Flutter (Dart, sin contratos compartidos); dos apps separadas (doble mantenimiento).
Consecuencias	Actualizar [PS] 1.4, la limitación de plataforma y RNF04 (píxeles CSS → dp). La distribución será por APK. Se mantienen las condiciones de RNF11 (primer plano). Hay que validar pronto las herramientas de Android (SP-A, SP-F).
Condición de revisión	Si los spikes muestran que MAUI no funciona de forma fiable en los dispositivos del equipo, o si el docente exige iOS o PWA.

ADR-02. Monolito modular con un único Servidor CAD

Elemento	Contenido
Contexto	RNF03 fija 20 sesiones; el equipo es de 3 personas; el plazo es corto y el presupuesto mínimo.
Decisión	Un proceso ASP.NET Core 10 con API, Hub SignalR, páginas Razor del panel y temporizador. Módulos por funcionalidad con capas ligeras.
Alternativas descartadas	API + SPA separada; servicios separados con broker.
Consecuencias	Despliegue y pruebas simples. Hace falta disciplina para respetar los límites entre módulos (revisión en PR).
Condición de revisión	Necesidad comprobada de escalar o desplegar partes por separado, o de más de una instancia.

ADR-03. PostgreSQL como fuente única de verdad y garante de la concurrencia

Elemento	Contenido
Contexto	RNF08 y los criterios de concurrencia de HU09, HU14–HU17, HU20, HU22–HU24, HU32, HU34 y HU41.
Decisión	PostgreSQL con transacciones por caso de uso, token xmin, índices únicos parciales e IdSolicitud. Sin PostGIS: latitud y longitud como double con CHECK de rango.
Alternativas descartadas	Redis o bloqueos en memoria (se pierden al reiniciar); MySQL (sin índices parciales); PostGIS (sin consultas espaciales requeridas).
Consecuencias	Las garantías no dependen del código de la interfaz; hay que traducir los conflictos a 409.
Condición de revisión	Si aparecen consultas espaciales de proximidad o un volumen que lo justifique.

ADR-04. SignalR solo para notificar; escrituras siempre por REST

Elemento	Contenido
Contexto	RNF02 exige notificar sin recargar; RNF06 y RNF08 exigen confirmar solo tras persistir y sin duplicados.
Decisión	Todas las escrituras van por REST con IdSolicitud; el Hub solo envía eventos después del COMMIT (y recibe acuses de medición). Los clientes se resincronizan por REST al reconectar.
Alternativas descartadas	Comandos por el Hub (idempotencia y errores más difíciles de manejar); sondeo periódico (no garantiza ≤3 s sin una carga alta).

Elemento	Contenido
Consecuencias	Separación clara entre persistido, confirmado y notificado. Los eventos perdidos se compensan con la resincronización.
Condición de revisión	Si se necesitara entrega garantizada de eventos sin consultar la base de datos.

ADR-05. Alojamiento Render + Neon con plan sin suspensión solo en la validación

Elemento	Contenido
Contexto	Presupuesto mínimo [EQ]; Render Free se suspende tras 15 min y la PostgreSQL gratuita de Render caduca a los 30 días [DOC]; RNF02 y RNF10 no toleran la suspensión.
Decisión	Desarrollo con Render Web Service Free (Docker) y Neon Free. Validación y demostraciones con Render Starter (prorrateo por segundo). Reglas temporales con `vence_en` + temporizador + comprobación al acceder. /salud sin acceso a la base de datos.
Alternativas descartadas	Solo capas gratuitas con *ping* (Render puede reiniciar el servicio; incertidumbre en RNF10); Azure for Students (más configuración); entorno local con túnel (aceptable por [EQ], pero menos representativo: queda como contingencia).
Consecuencias	Costo de hasta USD 7/mes prorrateado (más USD 6/mes si se cambia a Render Postgres). Hay que vigilar la cuota de Neon.
Condición de revisión	Si Neon agota la cuota o su arranque en frío afecta a RNF02, o si cambian los precios o las condiciones de los proveedores.

11. Plan de implementación

Hecho documentado: No hay código. [PS] planifica 181 SP en cinco sprints desde el 07/09/2026, pero a 04/10/2026 no existe ningún incremento: quedan 25 días naturales hasta la entrega objetivo del 29/10/2026. Este plan **no** inventa sprints, fechas ni capacidad. Ordena el trabajo por dependencias y riesgo y lo relaciona con el backlog existente. Replanificar el alcance (por ejemplo, qué hacer con los Could HU29, HU32 y HU33) corresponde al Product Owner.

11.1. Estructura inicial de carpetas

La estructura refleja la sección 3: un proyecto servidor con módulos y capas ligeras, un proyecto de contratos compartido, la app MAUI y dos proyectos de prueba. Las carpetas `Dominio` no referencian EF Core ni ASP.NET (sección 3.5).

```
cad/
├─ src/
│  ├─ Cad.Server/                    ASP.NET Core 10 (única unidad desplegable de servidor)
│  │  ├─ Program.cs                  composición: servicios, autenticación, endpoints, hub
│  │  ├─ Compartido/                 errores→HTTP, políticas de rol, reloj, extensiones
│  │  ├─ Persistencia/               CadDbContext, Configuraciones/, Migraciones/
│  │  ├─ TiempoReal/                 CadHub.cs, NotificadorSignalR.cs (implementa INotificador)
│  │  ├─ Temporizador/               TemporizadorDominio.cs (BackgroundService)
│  │  ├─ Modulos/
│  │  │  ├─ Cuentas/    {Dominio, Aplicacion, Endpoints}
│  │  │  ├─ Casos/      {Dominio, Aplicacion, Endpoints}
│  │  │  ├─ Despacho/   {Dominio, Aplicacion, Endpoints}
│  │  │  ├─ Acompanamiento/ {Dominio, Aplicacion, Endpoints}
│  │  │  └─ Consultas/  {Aplicacion, Endpoints}      (solo lectura)
│  │  ├─ Panel/Pages/                Razor Pages: Central, Casos, Acompanamientos, Admin/Cuentas
│  │  └─ wwwroot/                    js/mapa-operativo.js, js/tiempo-real.js, lib/leaflet, lib/signalr
│  ├─ Cad.Contratos/                 DTO, nombres de eventos y enumeraciones públicas (servidor + app)
│  └─ Cad.Movil/                     .NET MAUI 10 (Android)
│     ├─ Vistas/Ciudadano/  Vistas/Sereno/  Vistas/Comun/
│     ├─ VistasModelo/               MVVM
│     └─ Servicios/                  ApiCliente, TiempoRealCliente, UbicacionServicio, ColaPendiente
├─ tests/
│  ├─ Cad.Server.Tests/              dominio (unitarias) + integración con PostgreSQL real
│  └─ Cad.PruebaCarga/               consola: RNF02, RNF03 y monitor de RNF10
├─ docs/  adr/  diagramas/  despliegue.md  pruebas.md
├─ docker-compose.yml                PostgreSQL local para desarrollo y pruebas
├─ Dockerfile                        imagen del Servidor CAD
├─ .env.example                      variables sin valores reales
└─ .github/workflows/ci.yml          compilar + pruebas (servicio postgres)
```

11.2. Primer flujo completo de extremo a extremo

"Un ciudadano emite un SOS desde la app y el operador lo ve en la sección urgente, con sonido, en ≤3 s, en el entorno publicado." Cubre TEC01, HU01, HU02, HU03 (base), HU04 (o la semilla de un operador), HU06, HU07, HU08 y HU11, y valida de una vez la arquitectura completa: app → API → transacción → SignalR → panel, más el despliegue con HTTPS/WSS. Coincide con el objetivo del Sprint 1 de [PS].

11.3. Orden de construcción por dependencias y riesgo

Tabla 29. Orden de construcción propuesto

Paso	Contenido	Backlog [PS]	Riesgo que reduce	Depende de
0. Base desplegable	Repositorio, solución, CI, Dockerfile, Render + Neon, /salud, migración inicial, semilla del administrador (DP02), página "hola" publicada con HTTPS	TEC01	Despliegue tardío; secretos; WebSockets en Render	—
1. Spikes técnicos	SP-A … SP-F (sección 11.4)	SPIKE01 (reformulado para app nativa)	Herramientas MAUI, geolocalización, reconexión, arranque en frío	Paso 0
2. Flujo E2E mínimo	Registro e inicio de sesión (cookie y JWT), políticas de rol, permiso de ubicación, SOS idempotente con o sin ubicación, sección urgente con sonido, acuse y medición de RNF02, mapa operativo base (D3)	HU01, HU02, HU03 (construcción), HU04, HU06, HU07, HU08, HU11	Arquitectura completa sin validar	Pasos 0–1
3. Ciclo de despacho	Estado operativo y posición del sereno, disponibilidad, asignación concurrente, despacho, desplazamiento, inicio, cierre, verificación, clasificación con catálogo (DP07)	HU18, HU19, HU20-A, HU21–HU24, HU12, HU13	Concurrencia (RNF08); tiempo real hacia la app	Paso 2
4. Reportes y seguimiento	Reporte con mapa o texto, filtros, revisión, admisión, derivación, descarte, historial, mis casos, notificaciones persistidas	HU10, HU26, HU14–HU17, HU20-B, HU27, HU30, HU31	Visibilidad por rol (RNF12, DP05)	Paso 3
5. Acompañamiento	Sesión de 30 min, puntos cada 15 s, finalización con cola local, vencimiento en el servidor, consulta del operador, historial de sesiones, desactivación de cuentas	HU34–HU37, HU38, HU39-A, HU05 (HU39-B bloqueada por DP22)	Privacidad (RNF07, RNF11); reglas temporales	Pasos 2–3
6. Should / Could	Cancelación, bienestar, avisos, tiempos, evaluación, mapa de calor	HU09, HU40–HU43, HU28, HU32, HU33, HU29	Cláusulas pendientes (HU27-CA02, HU17-CA04)	Pasos 3–5
7. Validación	Cambio a Starter; pruebas de RNF02, RNF03, RNF04, RNF05 y RNF10; registro de pendientes	Fase 3 de [PS]	Incumplir RNF por la suspensión	Pasos 2–6

11.4. Pruebas técnicas breves (spikes)

Tabla 30. Spikes para resolver incertidumbres

ID	Pregunta	Criterio de salida
SP-A	¿La API de geolocalización de MAUI obtiene posición, maneja permiso denegado y respeta el primer plano en los Android del equipo, con *mock locations*?	Nota con resultados por dispositivo; decide qué puede afirmar la interfaz (RNF11)
SP-B	¿El cliente .NET de SignalR en MAUI se conecta a Render con WSS y reconecta tras un modo avión o una suspensión?	Reconexión y resincronización demostradas
SP-C	¿Mapsui muestra teselas OSM con User-Agent propio y permite seleccionar un punto (HU10-CA03)?	Pantalla de prueba funcionando
SP-D	¿Cuánto tarda el primer acceso tras la suspensión de Render Free y tras el escalado a cero de Neon?	Tiempos medidos; confirma o descarta Neon para la validación
SP-E	¿Suena el aviso del panel en Chrome y Firefox con la política de reproducción automática?	Procedimiento "Iniciar turno" validado
SP-F	¿Compila e instala el APK en los equipos y teléfonos de los tres integrantes?	APK instalado en al menos un teléfono de cada integrante

11.5. Pruebas mínimas de aceptación y de requisitos no funcionales

Tabla 31. Pruebas de requisitos no funcionales

RNF	Prueba	Herramienta	Evidencia
RNF01	Inspeccionar el código, la base de datos (columna de hash) y los logs tras el registro y el inicio de sesión	Revisión + consulta SQL + búsqueda en logs	Informe con la configuración de PasswordHasher
RNF02	≥100 SOS con 20 sesiones; un panel real abierto envía acuses	Cad.PruebaCarga + tabla MedicionPresentacion	% de SOS con aceptado→acuse ≤3 s
RNF03	30 min con SOS, reportes, asignaciones, cambios de estado y ≥5 acompañamientos	Cad.PruebaCarga + consultas SQL de consistencia	Sin dobles asignaciones, sin sesiones duplicadas, sin operaciones confirmadas perdidas
RNF04	Flujos del ciudadano y del sereno con anchos de 360, 390 y 412 dp	Emulador Android o dispositivos	Capturas por ancho (requiere actualizar el RNF: dp en lugar de px CSS)
RNF05	Funciones del panel en Chrome y Firefox fijados, a 1366×768 y 1920×1080	Lista de comprobación manual	Tabla de versiones y resultados
RNF06	SOS con un toque, con y sin ubicación; confirmación solo tras la respuesta	Prueba manual + red cortada	Vídeo o capturas
RNF07	Abrir la app no transmite; solo SOS, reporte por elección o sesión activa	Contadores por usuario de los endpoints de posición	Registro sin transmisiones indebidas
RNF08	Solicitudes paralelas y repetidas en cada operación crítica	xUnit + PostgreSQL real	Pruebas en verde en CI
RNF09	Solo HTTPS/WSS; intento sin cifrar rechazado	Inspección + app sin texto plano	Capturas de la configuración
RNF10	8 h de monitor: cada 30 s inicio de sesión + estado del panel; cada 5 min registro de un reporte de prueba	Cad.PruebaCarga --monitor sobre Starter	Disponibilidad = tiempo disponible / 8 h × 100
RNF11	Pérdida de conexión, permiso revocado, posiciones antiguas	Prueba manual con la app visible	Capturas de los estados de la interfaz y advertencias del panel
RNF12	Solicitudes directas por rol y por titularidad	Matriz automatizada (HU03-CA04)	Informe acumulativo por rol
RNF13	Eliminación con un plazo de prueba	Parámetro de prueba	Bloqueado por DP22
RNF14	Ramas, PR y CI	GitHub	Historial del repositorio

11.6. Despliegue temprano

- El paso 0 termina con el Servidor CAD publicado en Render (HTTPS) y conectado a Neon antes de escribir cualquier funcionalidad.
- Cada fusión en `main` despliega automáticamente; las pruebas de CI deben estar en verde antes de fusionar.
- El APK se genera y se comparte en cada hito, y la URL del servidor se configura en la app.
- Antes de cada demostración: calentar el servicio (o tenerlo en Starter) y comprobar `/salud/lista`.

11.7. Documentación mínima de ejecución y mantenimiento

Tabla 32. Documentación mínima

Documento	Contenido
README.md	Requisitos (.NET SDK 10, carga de trabajo de MAUI para Android, Docker), cómo arrancar PostgreSQL local, ejecutar el servidor, compilar el APK y apuntarlo a un servidor
.env.example	Nombres de las variables (`ConnectionStrings__Cad`, `Jwt__ClaveFirma`, `Admin__Email`, `Admin__ClaveInicial`, parámetros de temporizador y retención) sin valores reales
docs/despliegue.md	Alta en Render y Neon, región, variables, cambio de plan Free ↔ Starter, cómo revertir un despliegue
docs/pruebas.md	Cómo ejecutar las pruebas de RNF02, RNF03 y RNF10 y dónde quedan los informes
docs/adr/	ADR-01 … ADR-05 de este documento y los que se añadan
docs/diagramas/	Fuentes Mermaid y PlantUML (Anexo B) y las imágenes generadas

12. Auditoría de sobreingeniería

Tabla 33. Revisión de cada elemento del diseño

Elemento	Necesidad concreta	Si se elimina...	Alternativa más sencilla	¿Costo justificado?	Clasificación
ASP.NET Core 10	Servidor HTTP, autenticación, hosting	No hay servidor	—	Sí: el equipo conoce .NET	Imprescindible
SignalR	RNF02, HU11-CA02, HU21-CA01	Sondeo con una carga alta y sin garantía de ≤3 s	WebSocket propio (más código)	Sí: incluido en el framework	Imprescindible
Razor Pages + JS	Panel (RF05, RNF05)	No hay panel	Es ya la opción más simple con C#	Sí	Imprescindible
Leaflet + teselas OSM	RF05.2, HU19, HU25	Sin mapa operativo	—	Sí: libre	Imprescindible
Leaflet.heat	HU29 (Could)	Círculos con intensidad	Marcadores agrupados	Bajo	Opcional
.NET MAUI	App nativa pedida por [EQ]	Volver a la PWA	PWA (descartada por [EQ])	Sí, sujeto a los spikes	Imprescindible
Mapsui	HU10-CA03 (punto en el mapa), HU21	Solo referencia textual (incumple CA03)	WebView con Leaflet	Sí	Imprescindible
Cad.Contratos	DTO comunes entre servidor y app	DTO duplicados y desalineados	Copiar clases	Sí: un csproj	Recomendable
PostgreSQL	RNF08 (índices parciales, transacciones)	Sin garantías de concurrencia	SQLite (no apto en Render sin disco)	Sí: gratuito	Imprescindible
EF Core + migraciones	Acceso a datos, xmin, esquema versionado	SQL manual y scripts	Dapper	Sí	Imprescindible
Identity + cookie + JWT	RNF01, RF01, RF02; panel y app	Implementar hash y sesiones a mano (riesgo)	Un solo esquema (cookie también en la app)	Sí	Imprescindible
Temporizador en el proceso	HU37, HU42, RNF13	Las reglas temporales no se ejecutan	Solo evaluación diferida (no genera avisos)	Sí	Imprescindible
INotificador (interfaz)	Probar los casos de uso sin SignalR	La aplicación depende del Hub	Llamar a IHubContext directamente	Sí: una interfaz	Recomendable
Docker	Render no tiene runtime nativo de .NET	No se puede desplegar en Render	—	Sí: un Dockerfile	Imprescindible
Render Starter	RNF02 y RNF10 sin suspensión	Incumplir RNF10	Entorno local con túnel	Sí: USD 7/mes prorrateado y temporal	Imprescindible en validación
Neon	Base de datos sin caducidad a 30 días	Render Postgres Free caduca	Render Postgres de pago	Sí: gratuito	Imprescindible (o su alternativa)
GitHub Actions	RNF14; detectar errores pronto	Errores tardíos	Pruebas locales manuales	Sí: gratuito	Recomendable
Cad.PruebaCarga	Evidencia de RNF02, RNF03 y RNF10	No hay medición reproducible	k6 (sin SignalR nativo)	Sí	Imprescindible para validar
Health checks	RNF10; diagnóstico	El monitor no distingue base de datos y servidor	Endpoint propio	Sí: incluido	Recomendable
Limitador de inicio de sesión	Abuso de credenciales	Más riesgo de fuerza bruta	—	Bajo: incluido	Opcional
MedicionPresentacion	Medir RNF02 en el panel real	La medición no incluye la presentación	Medir solo en la herramienta	Sí: una tabla	Solo para pruebas

12.1. Elementos eliminados del diseño

Por no estar justificados por ningún requisito verificado: microservicios; API gateway; Redis o backplane de SignalR; broker de mensajes u outbox; CQRS y event sourcing; PostGIS; motor de búsqueda; varias bases de datos; Kubernetes; plataforma de observabilidad; notificaciones push (FCM); servicio de ubicación en segundo plano; patrón repositorio por entidad; MediatR y AutoMapper; SPA separada; Testcontainers (se usa PostgreSQL de Docker Compose y el contenedor de servicio de CI); app para iOS; segunda app para el sereno.

12.2. Comprobaciones de coherencia

Tabla 34. Comprobaciones finales

Comprobación	Resultado
El stack soporta las interacciones dibujadas	Sí: REST (HttpClient/fetch), SignalR (cliente .NET en MAUI y cliente JS en el panel), Npgsql/TLS y teselas HTTPS aparecen en C2, C3, en las secuencias y en el despliegue.
Los diagramas coinciden entre sí	Mismos nombres en C1–C3, secuencias y despliegue: App móvil CAD, Panel web de central, Servidor CAD, Base de datos CAD. Los componentes de C3 coinciden con los módulos de la sección 3.3 y con las carpetas de la 11.1.
Las dependencias de carpetas coinciden con las capas	Sí: Modulos/*/{Dominio, Aplicacion, Endpoints}, Persistencia (infraestructura) y TiempoReal (presentación + implementación de INotificador), como en la Figura 10.
No hay servicios ni bases de datos inventados	Una sola unidad de servidor y una sola base de datos; MedicionPresentacion es una tabla de prueba, no un almacén nuevo.
Las limitaciones técnicas no contradicen los requisitos	Primer plano y conexión como condición validada (RNF11, [PS] 1.5); sin promesas de seguimiento en segundo plano; RNF10 solo con Starter; RNF13 bloqueado por DP22 sin inventar plazos.
Las decisiones del equipo se respetan	C#/.NET, presupuesto mínimo, app nativa en Android, panel en Chrome (Firefox se mantiene por RNF05 hasta que el equipo decida, D12).

Anexo A. Correspondencia de requisitos funcionales entre documentos

Correspondencia propuesta **por contenido** entre la numeración agrupada de [PS] y la numeración plana usada en [HU]. No modifica ningún documento; conviene que el equipo la confirme y actualice [HU].

Tabla 35. Correspondencia RF [PS] ↔ RF [HU] ↔ HU

RF en [PS]	Nombre en [PS]	RF en [HU]	HU
RF01.1	Registro de ciudadano	RF01	HU01
RF01.2	Inicio de sesión	RF02	HU02
RF02	Control de acceso por roles	RF03	HU03
RF03.1	Emisión de SOS urgente	RF04	HU07
RF03.2	Ubicación de la alerta SOS	RF05	HU08
RF03.3	Cancelación de SOS pendiente	RF21	HU09
RF04	Preparación inicial de geolocalización	RF22	HU06
RF05.1	Recepción prioritaria del SOS	RF06	HU11
RF05.2	Visualización geográfica de casos	RF07	HU25
RF06	Transmisión de ubicación del sereno	RF08	HU18
RF07	Monitoreo y disponibilidad de serenos	RF09	HU19 (y parte de HU18, HU24)
RF08	Contacto y verificación del SOS	RF10	HU12
RF09	Asignación de caso	RF11	HU20 (A/B)
RF10.1	Recepción de despacho	RF12	HU21
RF10.2	Confirmación de desplazamiento	RF13	HU22
RF11.1	Registro de inicio de atención	RF14	HU23
RF11.2	Cierre de caso	RF15	HU24
RF12.1	Historial de casos	RF16 (parcial)	HU27
RF12.2	Búsqueda y filtrado de casos	RF19	HU26
RF13	Cálculo de tiempos de atención	RF17	HU28
RF14	Mapa de concentración de incidencias	RF18	HU29
RF15	Administración de cuentas institucionales	RF20	HU04, HU05
RF16.1	Registro de incidencias	RF23	HU10
RF16.2	Clasificación de incidencias	RF24	HU10, HU13, HU14
RF17	Revisión y resolución de reportes	RF25	HU14–HU17
RF18.1	Consulta de casos propios	RF26	HU30
RF18.2	Notificación de avances	RF27	HU31
RF19	Evaluación de atención	RF28	HU32, HU33
RF20.1	Activación del acompañamiento temporal	RF29	HU34
RF20.2	Finalización del acompañamiento	RF31	HU36, HU37
RF21	Transmisión y visualización del acompañamiento	RF30	HU35, HU38

RF en [PS]	Nombre en [PS]	RF en [HU]	HU
RF22	Confirmación de bienestar	RF32	HU40, HU41
RF23.1	Señales de verificación del acompañamiento	RF33	HU42
RF23.2	Revisión de avisos del acompañamiento	RF34	HU43
— (D14)	Sin equivalente explícito: historial de sesiones de acompañamiento	RF16 (parcial)	HU39 (A/B)

Anexo B. Código fuente de los diagramas

Cada figura se reproduce con su fuente. Renderizado: **Mermaid CLI 11.16** (mmdc, Chromium) con la configuración `mermaid-config.json` (fuente Arial 17 px, `useMaxWidth: false`) y **PlantUML 1.2025.4** con la biblioteca C4 incluida y Graphviz (`-Sdpi=200`). Comandos: `mmdc -i archivo.mmd -o archivo.png -c mermaid-config.json -s 2.5 -b white` y `java -jar plantuml.jar -tpng -Sdpi=200 archivo.puml`.

Figura 1. Proceso SOS · emisión y registro (Mermaid) — F01a_proceso_sos_emision.mmd

```
flowchart TB
  subgraph CIU["① Ciudadano · App móvil CAD"]
    C1(["Necesita ayuda urgente"]) --> C2["Pulsa SOS una sola vez<br/>(sin formulario) HU07"]
    C2 --> C3{"¿Posición<br/>actual? HU08"}
    C3 -->|Sí| C4["Adjunta coordenadas,<br/>hora y precisión"]
    C3 -->|No| C5["Indica 'Ubicación<br/>no disponible'"]
    C4 --> C6["Envía solicitud con<br/>IdSolicitud único"]
    C5 --> C6
    C6 -. "sin respuesta" .-> C8["Muestra 'No confirmado' y reintenta<br/>con el mismo IdSolicitud"]
  end

  subgraph SRV["② Servidor CAD"]
    S1{"¿Solicitud<br/>válida?"}
    S1 -->|No| SX["Responde rechazo:<br/>la app no confirma"]
    S1 -->|Sí| S2{"¿IdSolicitud<br/>repetido?"}
    S2 -->|Sí| S4["Devuelve el SOS existente<br/>(sin duplicar) RNF08"]
    S2 -->|No| S3["Registra SOS 'Pendiente'<br/>en una transacción"]
    S3 --> S9["Responde éxito: solo ahora la app<br/>muestra 'SOS registrado' RNF06"]
    S4 --> S9
    S3 --> S5(["Publica 'SOS nuevo' a operadores<br/>(continúa en la Figura 2)"])
  end

  C6 --> S1
```

Figura 2. Proceso SOS · recepción, despacho y atención (Mermaid) — F01b_proceso_sos_despacho.mmd

```
flowchart TB
  subgraph OPE["③ Operador · Panel web de central"]
    O0(["SOS nuevo publicado<br/>por el servidor (Figura 1)"]) --> O1["Ve el SOS en la sección urgente<br/>con aviso visual y sonoro HU11"]
    O1 --> O2["Contacta y registra intentos<br/>(opcional, no bloquea) HU12"]
    O2 --> O3["Clasifica el tipo<br/>(opcional, no bloquea) HU13"]
    O1 --> O4{"¿Sereno<br/>disponible?<br/>HU19"}
    O3 --> O4
    O4 -->|No| OW["Espera un cambio de disponibilidad;<br/>el caso sigue 'Pendiente'"]
    OW --> O4
    O4 -->|Sí| O5["Selecciona caso y sereno<br/>en el mapa y asigna HU20"]
    O5 --> O6{"¿Servidor<br/>acepta?"}
    O6 -->|"No: caso o sereno<br/>ya tomado"| O4
  end

  subgraph SER["④ Sereno · App móvil CAD"]
    E1["Recibe y consulta<br/>el despacho HU21"] --> E2["Confirma desplazamiento<br/>'En camino' HU22"]
    E2 --> E3["Registra inicio 'En atención' HU23<br/>(también puede hacerlo el operador)"]
    E3 --> E4["Cierra con tipificación y resultado<br/>'Cerrada' HU24 (o el operador)"]
    E4 --> E5(["Vuelve a figurar disponible"])
  end

  O6 -->|Sí| E1
```

Figura 3. Proceso de reporte de incidencias (Mermaid) — F02_proceso_reporte.mmd

```
flowchart TB
  subgraph CIU["① Ciudadano · App móvil CAD"]
    C1(["Quiere reportar un hecho"]) --> C2["Abre el formulario, que advierte<br/>usar SOS ante peligro inmediato HU10"]
    C2 --> C3["Indica tipo, descripción, fecha y hora<br/>aproximadas y ubicación del hecho:<br/>punto en mapa, referencia textual o<br/>posición actual solo si la elige RNF07"]
    C3 --> C8["Envía con IdSolicitud"]
  end

  subgraph SRV["② Servidor CAD"]
    S1{"¿Datos<br/>válidos?"}
    S1 -->|No| S3["Rechaza sin crear caso"]
    S1 -->|Sí| S2["Registra caso origen 'Reporte', estado<br/>'Recibido' y categoría original"]
  end

  subgraph OPE["③ Operador · Panel web de central"]
    O2["Ve el reporte en el listado (no en<br/>la sección urgente) e inicia revisión HU14"]
    O2 --> S4{"¿Sigue<br/>'Recibido'?"}
    S4 -->|No| S5["Rechazo: otro operador<br/>ya lo tomó"]
    S4 -->|Sí| O3["'En revisión'; corrige la categoría<br/>si corresponde (conserva la original)"]
```

```
    O3 --> O5{"Decisión"}
    O5 -->|Admitir| O6["'Pendiente' HU15"]
    O5 -->|Derivar| O7["Destino y acción:<br/>'Derivada' HU16"]
    O5 -->|Descartar| O8["Motivo obligatorio:<br/>'Descartada' HU17"]
    O6 --> O9["Asignación con las mismas<br/>reglas del SOS HU20 (Figura 2)"]
  end

  C8 --> S1
  S2 --> O2
```

Figura 4. Acompañamiento · activación y transmisión (Mermaid) — F03a_acompanamiento_inicio.mmd

```
flowchart TB
  subgraph CIU["① Ciudadano · App móvil CAD (en primer plano)"]
    C2["Pulsa 'Acompañamiento'<br/>(control independiente) HU34"] --> C3{"¿Acepta la<br/>finalidad?"}
    C3 -->|No| CX1(["No se crea sesión"])
    C3 -->|Sí| C4{"¿Permiso y<br/>posición?"}
    C4 -->|No| CX2(["No se crea sesión<br/>ni se transmite"])
    C4 -->|Sí| C5["Solicita inicio<br/>con IdSolicitud"]
    C6["Muestra estado activo, tiempo<br/>restante y problemas RNF11"]
    C6 --> C7["Cada 15 s intenta enviar la última<br/>posición con hora y precisión HU35"]
    C7 -->|"hasta finalizar o vencer<br/>(Figura 5)"| C7
  end

  subgraph SRV["② Servidor CAD"]
    S1{"¿Sesión activa<br/>existente?"}
    S1 -->|Sí| S3["Rechaza: una sola<br/>sesión activa RNF08"]
    S1 -->|No| S2["Crea sesión 'Activa' con<br/>vence_en = inicio + 30 min"]
    S4{"¿Activa y<br/>antes de vence_en?"}
    S4 -->|Sí| S5["Guarda el punto y lo<br/>publica a operadores"]
    S4 -->|No| S6["Rechaza el punto tardío"]
  end

  subgraph OPE["③ Operador · Panel web de central"]
    O1["Consulta sesiones activas: posición,<br/>hora, precisión y antigüedad HU38"]
  end

  C5 --> S1
  S2 --> C6
  C7 --> S4
  S5 --> O1
```

Figura 5. Acompañamiento · finalización y vencimiento (Mermaid) — F03b_acompanamiento_fin.mmd

```
flowchart TB
  subgraph CIU["① Ciudadano · App móvil CAD"]
    C8{"¿Decide<br/>finalizar?"}
    C8 -->|Sí| C9["Detiene captura y<br/>envío de inmediato HU36"]
    C9 --> C10{"¿Hay<br/>conexión?"}
    C10 -->|Sí| C12["Envía cierre<br/>(IdSolicitud)"]
    C10 -->|No| C11["Guarda el cierre pendiente<br/>y lo reintenta (efecto: DP20)"]
    C11 -->|al reconectar| C12
    C13["Detiene y muestra<br/>'Finalizado por tiempo'"]
  end

  subgraph SRV["② Servidor CAD"]
    S8["Registra 'Finalizada'<br/>(idempotente)"]
    S7["Al llegar a vence_en marca 'Vencida'<br/>aunque el dispositivo esté desconectado HU37"]
    S9["Rechaza posiciones posteriores<br/>al cierre o vencimiento"]
    S10["No genera aviso de peligro ni<br/>cancela un SOS existente"]
  end

  C8 -->|No| CX(["Sigue transmitiendo<br/>(Figura 4)"])
  C12 --> S8
  S8 --> S9
  S7 --> S9
  S8 --> S10
  S7 --> S10
  S7 -. "si hay conexión o<br/>al reconectar" .-> C13
```

Figura 6. Confirmación de bienestar (Mermaid) — F04a_bienestar.mmd

```
flowchart TB
  subgraph OPE["① Operador · Panel web de central"]
    O1["Solicita confirmación de bienestar<br/>sobre una sesión activa HU40"]
  end
  subgraph SRV["② Servidor CAD"]
    S1{"¿Sesión<br/>activa?"}
    S1 -->|No| S3["Rechaza"]
    S1 -->|Sí| S2["Registra la solicitud<br/>(no crea SOS ni despacho)"]
    S8{"¿SOS vigente<br/>del ciudadano?<br/>(ver D9)"}
    S8 -->|Sí| S9["Vincula la respuesta<br/>con ese SOS"]
    S8 -->|No| S10["Inicia el flujo SOS<br/>HU07/HU08 (idempotente)"]
    S11["Sin respuesta en 2 min:<br/>genera aviso (Figura 7)"]
  end
```

```
  subgraph CIU["③ Ciudadano · App móvil CAD"]
    C1["Ve la solicitud; su<br/>presentación se registra HU41"] --> C2{"Respuesta"}
    C2 -->|Estoy bien| C3["Se registra;<br/>la sesión continúa"]
    C2 -->|Necesito ayuda| C4["Envía 'Necesito ayuda'<br/>con IdSolicitud"]
  end
  O1 --> S1
  S2 --> C1
  C4 --> S8
  C1 -. "sin respuesta" .-> S11
```

Figura 7. Avisos del acompañamiento (Mermaid) — F04b_avisos.mmd

```
flowchart TB
  subgraph SRV["① Servidor CAD"]
    S4["Temporizador: evalúa reglas solo<br/>en sesiones activas HU42"]
    S4 --> S5{"¿Más de 60 s<br/>sin posiciones?"}
    S4 --> S6{"¿Confirmación<br/>sin respuesta<br/>tras 2 min?"}
    S5 -->|Sí| S7["Genera un aviso por condición<br/>(sin repetir) y lo publica"]
    S6 -->|Sí| S7
  end
  subgraph OPE["② Operador · Panel web de central"]
    O2["Ve el aviso separado de los SOS<br/>(sin despacho automático)"] --> O3["Revisa: 'En revisión' y registra
contactos,<br/>acciones y resultados HU43"]
    O3 --> O4{"¿Requiere<br/>un caso?"}
    O4 -->|Vincular| O5["Vincula con un<br/>caso existente"]
    O4 -->|Nuevo| O6["Crea caso urgente con<br/>justificación (DP19)"]
    O4 -->|No| O7["Resuelve:<br/>'Resuelto'"]
    O5 --> O7
    O6 --> O7
    O6 -.-> O8["Asignación manual<br/>con reglas de HU20"]
  end
  S7 --> O2
```

Figura 8. Ciclo de vida del caso (SOS y reporte) (Mermaid stateDiagram) — F05_estados_caso.mmd

```
stateDiagram-v2
  direction TB
  [*] --> Pendiente: SOS aceptado por el servidor (HU07)
  [*] --> Recibido: Reporte registrado (HU10)
  Recibido --> EnRevision: Iniciar revisión (HU14)
  EnRevision --> Pendiente: Admitir (HU15)
  EnRevision --> Derivada: Derivar con destino y acción (HU16)
  EnRevision --> Descartada: Descartar con motivo (HU17)
  Pendiente --> Cancelada: Cancelar con motivo, solo SOS propio (HU09)
  Pendiente --> Asignada: Asignar a sereno disponible (HU20)
  Asignada --> EnCamino: Sereno asignado confirma (HU22)
  EnCamino --> EnAtencion: Sereno asignado u operador (HU23)
  EnAtencion --> Cerrada: Tipificación y resultado (HU24)
  Cerrada --> [*]
  Cancelada --> [*]
  Derivada --> [*]
  Descartada --> [*]
  EnRevision: En revisión
  EnCamino: En camino
  EnAtencion: En atención
  note right of Asignada
    No definidas en los requisitos (no se dibujan):
    reasignación, rechazo del sereno, reapertura,
    efecto de DP16 (sereno que se desactiva) y DP03
    (cuenta desactivada con asignación vigente).
  end note
```

Figura 9. Estados de la sesión de acompañamiento, del aviso y del sereno (Mermaid stateDiagram) — F06_estados_acompanamiento.mmd

```
stateDiagram-v2
  direction LR
  state "Sesión de acompañamiento" as SA {
    [*] --> Activa: Inicio confirmado por el servidor (HU34)
    Activa --> Finalizada: El ciudadano finaliza (HU36)
    Activa --> Vencida: vence_en alcanzado, sin renovación (HU37)
    Finalizada --> [*]
    Vencida --> [*]
  }
  state "Aviso del acompañamiento" as AV {
    [*] --> Generado: Regla cumplida (HU42)
    Generado --> AvisoEnRevision: Operador revisa (HU43)
    AvisoEnRevision --> Resuelto: Operador resuelve (HU43)
    Resuelto --> [*]
    AvisoEnRevision: En revisión
  }
  state "Estado operativo del sereno" as SE {
    [*] --> NoOperativo
    NoOperativo --> Operativo: Activa su estado (HU18)
    Operativo --> NoOperativo: Desactiva (HU18) · con asignación vigente: DP16
    NoOperativo: No operativo
```

```
  }
```

Figura 10. Capas del Servidor CAD y dependencias de código (Mermaid) — F07_capas_dependencias.mmd

```
flowchart TB
  P["<b>Presentación</b><br/>Endpoints REST /api · Páginas Razor del panel<br/>Hub SignalR /hubs/cad y
NotificadorSignalR<br/>Temporizador (BackgroundService)<br/><i>Program.cs: composición y configuración</i>"]
  A["<b>Aplicación</b> · un servicio de casos de uso por módulo<br/>Cuentas · Casos · Despacho · Acompañamiento ·
Consultas<br/>Autorización por titularidad · transacciones · define INotificador"]
  D["<b>Dominio</b><br/>Entidades · estados permitidos · reglas e invariantes<br/><i>sin referencias a EF Core, ASP.NET
ni SignalR</i>"]
  I["<b>Infraestructura</b><br/>CadDbContext (EF Core + Npgsql, migraciones)<br/>Identity · emisión de JWT · reloj"]
  DB[("PostgreSQL")]

  P -->|"invoca casos de uso"| A
  A -->|"usa entidades y reglas"| D
  A -->|"usa CadDbContext<br/>(sin repositorios por entidad)"| I
  I -->|"mapea entidades"| D
  P -. "NotificadorSignalR implementa<br/>INotificador (inversión)" .-> A
  P -->|"registra servicios<br/>y autenticación"| I
  I -->|"SQL"| DB

  classDef capa fill:#EEF3FB,stroke:#2F5597,color:#0B1F3A
  classDef infra fill:#FFF6E5,stroke:#B26B00,color:#3A2300
  class P,A,D capa
  class I infra
```

Figura 11. Flujo de ejecución de una operación representativa: emitir un SOS (Mermaid) — F08_flujo_ejecucion_sos.mmd

```
flowchart TB
  U["App móvil<br/>POST /api/sos + JWT + IdSolicitud"] --> M["Middleware: HTTPS, autenticación,<br/>cuenta activa,
límite de tamaño"]
  M --> E["Endpoint EmitirSos: valida formato<br/>y política de rol Ciudadano"]
  E --> S["Casos.EmitirSos (aplicación):<br/>idempotencia y regla de ubicación"]
  S --> D["Dominio: Caso.CrearSos()<br/>estado 'Pendiente' +<br/>evento de historial"]
  D --> T[("Transacción única:<br/>caso + evento<br/>(SaveChanges)")]
  T --> R["Respuesta 201<br/>(o 200 si repetido)"]
  R --> U2["App muestra<br/>'SOS registrado'"]
  T --> N["INotificador.SosNuevo<br/>después del commit"]
  N --> H["Hub: grupo<br/>'operadores'"]
  H --> P["Panel: sección urgente,<br/>sonido y acuse<br/>de presentación"]
```

Figura 12. C1 · Contexto del sistema (C4-PlantUML) — C1_contexto.puml

```
@startuml C1_contexto
!include <C4/C4_Context>
LAYOUT_TOP_DOWN()
SHOW_PERSON_OUTLINE()
title C1 · Contexto del Sistema Integrado CAD (MVP académico, datos simulados)

Person(ciudadano, "Ciudadano", "Emite SOS, registra reportes, comparte su ubicación temporalmente y sigue sus casos")
Person(sereno, "Sereno", "Activa su estado operativo, comparte ubicación y atiende despachos")
Person(operador, "Operador de central", "Recibe, verifica, asigna y supervisa casos y acompañamientos")
Person(admin, "Administrador", "Gestiona cuentas de operadores y serenos")

System(cad, "Sistema Integrado CAD", "Alertas georreferenciadas, despacho manual asistido, seguimiento y acompañamiento
temporal")

System_Ext(osm, "Servidor de teselas OpenStreetMap", "tile.openstreetmap.org · mapas base, sin SLA (uso razonable)")
System_Ext(gps, "Servicios de ubicación de Android", "GPS/red del teléfono · solo con permiso y app en primer plano")

Rel(ciudadano, cad, "Emite SOS, reportes y acompañamientos; consulta sus casos", "App móvil · HTTPS/WSS")
Rel(sereno, cad, "Recibe despachos y registra la atención", "App móvil · HTTPS/WSS")
Rel(operador, cad, "Atiende y despacha casos", "Navegador · HTTPS/WSS")
Rel(admin, cad, "Administra cuentas institucionales", "Navegador · HTTPS")
Rel(cad, gps, "Solicita la posición actual", "API de ubicación del dispositivo")
Rel(cad, osm, "Descarga teselas de mapa", "HTTPS")

Lay_R(ciudadano, sereno)
Lay_R(sereno, operador)
Lay_R(operador, admin)

@enduml
```

Figura 13. C2 · Contenedores (C4-PlantUML) — C2_contenedores.puml

```
@startuml C2_contenedores
!include <C4/C4_Container>
LAYOUT_TOP_DOWN()
SHOW_PERSON_OUTLINE()
```

```
title C2 · Contenedores del Sistema Integrado CAD

Person(ciudadano, "Ciudadano")
Person(sereno, "Sereno")
Person(operador, "Operador de central")
Person(admin, "Administrador")

System_Boundary(cad, "Sistema Integrado CAD") {
  Container(app, "App móvil CAD", ".NET MAUI 10 · Android", "Modo ciudadano (SOS, reportes, acompañamiento, mis casos) y
modo sereno (estado operativo, despachos). Mapas con Mapsui")
  Container(panel, "Panel web de central", "HTML + JavaScript en Chrome/Firefox · Leaflet 1.9.4 · cliente SignalR",
"Sección urgente, mapa operativo, revisión, historial y administración de cuentas")
  Container(server, "Servidor CAD", "ASP.NET Core 10 · un solo proceso", "API REST, Hub SignalR, páginas Razor del
panel, autenticación y temporizador de dominio")
  ContainerDb(db, "Base de datos CAD", "PostgreSQL", "Fuente de verdad: cuentas, casos, historial, asignaciones,
sesiones, puntos, avisos y notificaciones")
}

System_Ext(osm, "Servidor de teselas OpenStreetMap", "tile.openstreetmap.org")
System_Ext(gps, "Servicios de ubicación de Android", "GPS/red")

Rel(ciudadano, app, "Usa")
Rel(sereno, app, "Usa")
Rel(operador, panel, "Usa")
Rel(admin, panel, "Usa")
Rel(app, server, "Comandos y consultas (JWT)", "HTTPS · JSON")
Rel(server, app, "Eventos: despacho, avances, cierre de sesión", "WSS · SignalR")
Rel(server, panel, "Entrega páginas y recursos (cookie de sesión)", "HTTPS")
Rel(panel, server, "Acciones del operador", "HTTPS · JSON + antifalsificación")
Rel(server, panel, "Eventos: SOS nuevo, posiciones, avisos", "WSS · SignalR")
Rel(server, db, "Lee y escribe en transacciones", "TCP/TLS · Npgsql")
Rel(app, gps, "Obtiene posición", "API de MAUI")
Rel(app, osm, "Teselas (User-Agent propio)", "HTTPS")
Rel(panel, osm, "Teselas", "HTTPS")
Lay_R(ciudadano, sereno)
Lay_R(sereno, operador)
Lay_R(operador, admin)
Lay_R(app, panel)
Lay_D(server, db)
Lay_R(gps, osm)

@enduml
```

Figura 14. C3 · Componentes del Servidor CAD (C4-PlantUML) — C3_componentes.puml

```
@startuml C3_componentes
!include <C4/C4_Component>
LAYOUT_TOP_DOWN()
title C3 · Componentes del Servidor CAD (ASP.NET Core 10)

Container(app, "App móvil CAD", ".NET MAUI 10")
Container(panel, "Panel web de central", "HTML + JS")
ContainerDb(db, "Base de datos CAD", "PostgreSQL")

Container_Boundary(server, "Servidor CAD") {
  Component(api, "Endpoints REST", "Minimal APIs /api/*", "Validan formato, aplican políticas de rol y traducen
resultados a HTTP (400/403/404/409)")
  Component(pages, "Páginas del panel", "Razor Pages", "Pantallas del operador y del administrador; inicio de sesión con
cookie")
  Component(hub, "Hub de tiempo real", "SignalR /hubs/cad + NotificadorSignalR", "Une cada conexión a grupos según rol e
identidad y envía eventos tras cada confirmación")
  Component(auth, "Cuentas y acceso", "ASP.NET Core Identity + JWT", "Registro, inicio de sesión, hash PBKDF2, cuentas
institucionales, revocación por sello de seguridad")
  Component(casos, "Casos", "Servicio de aplicación + dominio", "SOS, reportes, verificación, clasificación, revisión,
cancelación, historial y notificaciones al ciudadano")
  Component(despacho, "Despacho y serenos", "Servicio de aplicación + dominio", "Estado operativo, posiciones,
disponibilidad, asignación, desplazamiento, atención y cierre")
  Component(acomp, "Acompañamiento", "Servicio de aplicación + dominio", "Sesiones de 30 min, puntos, bienestar, avisos
y retención (DP22)")
  Component(consultas, "Consultas", "Servicio de solo lectura", "Filtros, tiempos, mapa de calor, evaluaciones y estado
inicial para resincronizar")
  Component(timer, "Temporizador de dominio", "BackgroundService", "Vence sesiones, evalúa reglas de avisos y ejecuta la
eliminación por plazo cuando esté definida")
  Component(dbctx, "CadDbContext", "EF Core 10 + Npgsql", "Mapeo, migraciones, restricciones únicas y token de
concurrencia xmin")
}

Rel(app, api, "Llama", "HTTPS/JSON + JWT")
Rel(panel, api, "Llama", "HTTPS/JSON + cookie")
Rel(panel, pages, "Navega", "HTTPS")
Rel(app, hub, "Se suscribe", "WSS")
Rel(panel, hub, "Se suscribe y envía acuses", "WSS")
Rel(api, auth, "Autentica y autoriza")
Rel(api, casos, "Invoca")
Rel(api, despacho, "Invoca")
Rel(api, acomp, "Invoca")
Rel(api, consultas, "Invoca")
Rel(pages, consultas, "Invoca")
Rel(pages, auth, "Invoca")
Rel(acomp, casos, "Crea o vincula SOS (bienestar, avisos)")
Rel(despacho, casos, "Cambia estados del caso")
Rel(timer, acomp, "Invoca cada pocos segundos")
```

```
Rel(casos, hub, "Publica eventos (INotificador)")
Rel(despacho, hub, "Publica eventos (INotificador)")
Rel(acomp, hub, "Publica eventos (INotificador)")
Rel(casos, dbctx, "Usa")
Rel(despacho, dbctx, "Usa")
Rel(acomp, dbctx, "Usa")
Rel(consultas, dbctx, "Consulta sin seguimiento")
Rel(auth, dbctx, "Usa")
Rel(dbctx, db, "SQL", "TCP/TLS")
@enduml
```

Figura 15. Secuencia: emisión del SOS, persistencia, confirmación y notificación en tiempo real (Mermaid) — S01_secuencia_sos.mmd

```
sequenceDiagram
  autonumber
  actor Ciu as Ciudadano
  participant App as App móvil CAD
  participant Srv as Servidor CAD (API + Hub)
  participant BD as PostgreSQL
  participant Pan as Panel web (operador)

  Note over Pan,Srv: El panel ya está conectado al Hub (WSS) en el grupo 'operadores'
  Ciu->>App: Pulsa SOS una vez
  App->>App: Intenta obtener la posición actual (tiempo límite corto)
  App->>Srv: POST /api/sos {IdSolicitud, ubicación o 'no disponible'} + JWT (HTTPS)
  Srv->>Srv: Autentica, comprueba cuenta activa y rol Ciudadano
  alt Token inválido, cuenta desactivada o datos inválidos
    Srv-->>App: 401/403/400 (nada se persiste)
    App-->>Ciu: "No confirmado" con opción de reintentar
  else Solicitud válida
    Srv->>BD: BEGIN · INSERT caso 'Pendiente' + evento · COMMIT
    alt IdSolicitud ya existía (reintento técnico)
      BD-->>Srv: violación de unicidad (ciudadano, IdSolicitud)
      Srv->>BD: SELECT del SOS existente
      Srv-->>App: 200 con el mismo SOS (sin duplicado)
    else Primera vez
      BD-->>Srv: COMMIT correcto (marca aceptado_en)
      Srv-->>App: 201 Creado {id, estado 'Pendiente'}
      Srv-)Pan: Evento 'SosNuevo' {id, ubicación, aceptado_en}
      Pan->>Pan: Sección urgente + sonido
      Pan-)Srv: Acuse 'Presentado' {id} (medición RNF02)
    end
    App-->>Ciu: "SOS registrado" (solo tras la respuesta del servidor)
  end
```

Figura 16. Secuencia: fallo de conexión y recuperación (Mermaid) — S03_secuencia_fallos.mmd

```
sequenceDiagram
  autonumber
  participant App as App móvil CAD
  participant Srv as Servidor CAD (API + Hub)
  participant BD as PostgreSQL
  participant Pan as Panel web (operador)

  rect rgb(245,245,245)
  Note over App,Srv: Caso A · la respuesta al POST /api/sos no llega (corte de red o servidor reiniciado)
  App->>App: No confirma el SOS · mantiene el IdSolicitud en almacenamiento local
  App->>Srv: POST /api/sos (reintento con el MISMO IdSolicitud)
  alt El primer intento sí se había persistido
    Srv->>BD: INSERT falla por unicidad → SELECT del SOS existente
    Srv-->>App: 200 con el SOS ya registrado (sin duplicado)
  else El primer intento no llegó a persistirse
    Srv->>BD: BEGIN · INSERT · COMMIT
    Srv-->>App: 201 Creado
    Srv-)Pan: Evento 'SosNuevo'
  end
  end

  rect rgb(245,245,245)
  Note over Srv,Pan: Caso B · el panel pierde la conexión WebSocket
  Pan->>Pan: Muestra "Sin tiempo real" · reconexión automática con espera creciente
  Srv--xPan: Eventos emitidos durante el corte: se pierden (no hay cola)
  Pan->>Srv: Reconecta (WSS) · se vuelve a unir a 'operadores' según su rol
  Pan->>Srv: GET /api/central/estado
  Srv->>BD: SOS 'Pendiente', casos activos, serenos, sesiones y avisos
  Srv-->>Pan: Estado completo · el panel redibuja (la BD es la fuente de verdad)
  end

  rect rgb(245,245,245)
  Note over App,Srv: Caso C · el servidor se suspende o reinicia
  Srv--xApp: Conexiones WebSocket cerradas
  App->>Srv: Reintentos de reconexión (≈1 min si el plan gratuito estaba suspendido)
  App->>Srv: GET de estado propio (asignación vigente, sesión activa, notificaciones)
  Note over Srv,BD: Las sesiones vencidas durante la suspensión se cierran al primer acceso (vence_en ≤ ahora)
  end
```

Figura 17. Secuencia: asignaciones concurrentes del mismo caso (Mermaid) — S02_secuencia_asignacion.mmd

```
sequenceDiagram
  autonumber
  participant OpA as Panel · Operador A
  participant OpB as Panel · Operador B
  participant Srv as Servidor CAD
  participant BD as PostgreSQL
  participant Ser as App móvil · Sereno S1

  par Dos asignaciones casi simultáneas del mismo caso C1
    OpA->>Srv: POST /api/casos/C1/asignacion {sereno: S1}
  and
    OpB->>Srv: POST /api/casos/C1/asignacion {sereno: S2}
  end
  Srv->>BD: Tx A: lee C1 (Pendiente, xmin=7) · UPDATE C1 'Asignada' WHERE xmin=7 · INSERT asignación vigente S1
  BD-->>Srv: Tx A COMMIT
  Srv->>BD: Tx B: UPDATE C1 'Asignada' WHERE xmin=7
  BD-->>Srv: 0 filas (xmin cambió) → conflicto · ROLLBACK
  Srv-->>OpB: 409 "El caso ya fue asignado" (sin registros parciales)
  Srv-->>OpA: 200 Asignado a S1
  Srv-)OpA: Evento 'CasoActualizado' + 'DisponibilidadSereno' (grupo operadores)
  Srv-)OpB: Evento 'CasoActualizado' + 'DisponibilidadSereno' (grupo operadores)
  Srv-)Ser: Evento 'DespachoAsignado' {C1} (grupo sereno:S1)
  Ser->>Srv: GET /api/sereno/asignacion-vigente (datos visibles para el sereno, DP06)
  Note over Srv,BD: Si dos casos distintos se asignan a S1 a la vez, el índice único parcial<br/>asignacion(sereno_id)
WHERE vigente rechaza el segundo → 409 "Sereno no disponible"
  Note over Ser,Srv: Si la app estaba desconectada, al reconectar consulta la asignación vigente (la BD es la fuente de
verdad)
```

Figura 18. Despliegue mínimo propuesto (C4-PlantUML, Deployment) — D01_despliegue.puml

```
@startuml D01_despliegue
!include <C4/C4_Deployment>
LAYOUT_TOP_DOWN()
title Despliegue mínimo del Sistema Integrado CAD

Deployment_Node(tel, "Teléfono Android", "Android · APK instalado manualmente") {
  Container(app, "App móvil CAD", ".NET MAUI 10", "Guarda el JWT en SecureStorage; URL del servidor configurable")
}
Deployment_Node(pc, "PC de la central", "Windows") {
  Deployment_Node(nav, "Navegador", "Chrome / Firefox (RNF05)") {
    Container(panel, "Panel web de central", "HTML + JS", "Leaflet, cliente SignalR")
  }
}
Deployment_Node(render, "Render · Web Service", "Región EE. UU. Este · Free en desarrollo, Starter en validación y
demostraciones") {
  Deployment_Node(docker, "Contenedor Docker", "imagen aspnet:10.0") {
    Container(server, "Servidor CAD", "ASP.NET Core 10", "Variables de entorno: ConnectionStrings~__Cad,
Jwt~__ClaveFirma, Admin~__*; HTTPS terminado por Render")
  }
}
Deployment_Node(neon, "Neon · proyecto PostgreSQL", "AWS us-east-1 · plan Free (alternativa: Render Postgres)") {
  ContainerDb(db, "Base de datos CAD", "PostgreSQL", "sslmode=require; escala a cero tras 5 min sin uso")
}
Deployment_Node(gh, "GitHub", "Repositorio + GitHub Actions") {
  Container(ci, "Flujo CI", "GitHub Actions", "Compila y prueba con un PostgreSQL de servicio; main despliega
automáticamente en Render")
}
System_Ext(osm, "Servidor de teselas OpenStreetMap", "tile.openstreetmap.org")

Rel(app, server, "API y eventos", "HTTPS + WSS (443)")
Rel(panel, server, "Páginas, API y eventos", "HTTPS + WSS (443)")
Rel(server, db, "SQL", "TCP 5432 sobre TLS")
Rel(ci, server, "Despliegue automático desde la rama main", "Git (integración de Render)")
Rel(app, osm, "Teselas", "HTTPS")
Rel(panel, osm, "Teselas", "HTTPS")
@enduml
```

Figura 19. Modelo de datos conceptual (Mermaid erDiagram) — F09_modelo_datos.mmd

```
erDiagram
  direction TB
  USUARIO ||--o| PERFIL_CIUDADANO : "si rol Ciudadano"
  USUARIO ||--o| ESTADO_SERENO : "si rol Sereno"
  USUARIO ||--o{ CASO : "emite (ciudadano)"
  CATEGORIA ||--o{ CASO : "clasifica"
  CASO ||--o{ EVENTO_CASO : "historial"
  CASO ||--o{ ASIGNACION : "tiene"
  USUARIO ||--o{ ASIGNACION : "sereno asignado"
  CASO ||--o| EVALUACION : "una por caso"
  USUARIO ||--o{ NOTIFICACION : "recibe"
  CASO ||--o{ NOTIFICACION : "origina"
  USUARIO ||--o{ SESION_ACOMPANAMIENTO : "inicia"
  SESION_ACOMPANAMIENTO ||--o{ PUNTO_ACOMPANAMIENTO : "registra"
  SESION_ACOMPANAMIENTO ||--o{ SOLICITUD_BIENESTAR : "recibe"
  SESION_ACOMPANAMIENTO ||--o{ AVISO : "genera"
```

```
  AVISO ||--o{ EVENTO_AVISO : "actuaciones"
  AVISO }o--o| CASO : "vinculado a"
  SOLICITUD_BIENESTAR }o--o| CASO : "SOS vinculado"
```

Anexo C. Documentación oficial consultada (04/10/2026)

Tabla 36. Fuentes externas

Tema	URL
Render: capa gratuita (suspensión, Postgres de 30 días, límites)	https://render.com/docs/free
Render: precios (Starter, Postgres, prorrateo)	https://render.com/pricing
Neon: límites del plan Free	https://neon.com/faqs/free-plan-limits-and-quotas
.NET: política de soporte	https://dotnet.microsoft.com/en-us/platform/support/policy
.NET MAUI: política de soporte	https://dotnet.microsoft.com/en-us/platform/support/policy/maui
Mapsui.Maui en NuGet	https://www.nuget.org/packages/Mapsui.Maui/
Npgsql.EntityFrameworkCore.PostgreSQL en NuGet	https://www.nuget.org/packages/Npgsql.EntityFrameworkCore.PostgreSQL
Leaflet: descargas y versión estable	https://leafletjs.com/download.html
OpenStreetMap: política de uso de teselas	https://operations.osmfoundation.org/policies/tiles/
ASP.NET Core Identity: PasswordHasher (código fuente)	https://source.dot.net/Microsoft.Extensions.Identity.Core/PasswordHasher.cs.html
Cloudflare Quick Tunnels (contingencia)	https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/do-more-with-tunnels/trycloudflare/